

EC200A-CN(TA) QuecOpen

网络管理应用指导

车规级模块系列

版本：1.0.0

日期：2022-07-15

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2022，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2022.

文档历史

修订记录

版本	日期	作者	变更表述
-	2022-07-15	Allen FENG	文档创建
1.0.0	2022-07-15	Allen FENG	临时版本

目录

文档历史	3
目录	4
表格索引	6
图片索引	7
1 引言	8
2 网络管理概念解释	9
3 QL_NF 介绍	12
3.1. QL_NF 规则配置	13
3.2. NF_INTF	14
3.2.1. NF_INTF 规则配置	14
3.2.2. NF_INTF 寻址方式	16
3.2.3. NF_INTF 状态	16
3.2.4. 创建 NF_INTF	17
3.2.5. NF_INTF 配置示例	19
3.3. VLAN 管理	19
3.4. 桥接管理	20
3.5. Wi-Fi 管理	21
3.6. LAN 管理	22
3.6.1. DHCP 服务管理	22
3.6.1.1. DHCP 配置示例	24
3.6.2. DNS 服务管理	25
3.6.3. LAN 管理示例	26
3.6.4. LAN 主机管理	26
3.7. 静态 ARP 规则管理	27
3.8. 数据拨号	28
3.8.1. 拨号介绍	28
3.8.2. 重连机制	29
3.8.3. 设置默认承载 APN	29
3.8.4. 数据拨号状态	30
3.8.5. 数据拨号命令	31
3.8.6. 拨号示例	33
3.8.7. 注意事项	33
3.9. 转发策略	33
3.9.1. 转发策略介绍	33
3.9.2. ACL 命令	34
3.10. QL_NF API	36
3.10.1. 头文件	36
3.10.2. 函数概览	36
3.10.3. 函数详解	37
3.10.3.1. ql_nf_init	37

3.10.3.2.	ql_nf_cmd.....	37
3.10.3.3.	ql_nf_set_interface_status_ind_cb	38
3.10.3.4.	ql_nf_set_service_error_cb.....	41
3.10.3.5.	ql_nf_deinit.....	42
3.11.	注意事项.....	42
3.11.1.	MAC 地址分配	42
3.11.2.	禁用 QL_NF	43
4	场景案例.....	44
4.1.	场景一.....	44
4.1.1.	场景需求.....	44
4.1.2.	QL_NF 组网.....	45
4.1.3.	规则配置.....	47
4.1.4.	场景测试.....	48
4.2.	场景二.....	48
4.2.1.	场景需求.....	48
4.2.2.	QL_NF 组网.....	49
4.2.3.	规则配置.....	51
4.2.4.	场景测试.....	52
4.3.	场景三.....	53
4.3.1.	场景需求.....	53
4.3.2.	QL_NF 组网.....	54
4.3.3.	规则配置.....	56
4.3.4.	场景测试.....	58
4.3.4.1.	模块配置.....	59
4.3.4.2.	Wi-Fi 配置(wifi 接口部分目前接口暂时未开发).....	60
4.3.4.3.	PC1 配置.....	61
4.3.4.4.	PC2 配置.....	61
4.3.4.5.	数据转发测试.....	61
4.4.	场景四.....	62
4.4.1.	场景需求.....	62
4.4.2.	QL_NF 组网.....	64
4.4.3.	规则配置.....	66
5	附录 参考文档及术语缩写	69

表格索引

表 1: 路由器相关配置 9

表 2: NF_INTF 配置参数 15

表 3: NF_INTF 状态参数 16

表 4: NF_INTF 类型 (<INTERFACE_TYPE>) 和关键参数 (<KEY_PARAMS>) 17

表 5: NF_INTF 关键参数 (<KEY_PARAMS>) 描述 18

表 6: 桥接接口扩展配置参数 21

表 7: 配置参数 24

表 8: 状态参数 24

表 9: DNS 服务状态参数 26

表 10: LAN 主机信息 27

表 11: ARP 规则参数列表 27

表 12: APN 参数列表 29

表 13: 数据拨号配置参数 32

表 14: 状态参数 32

表 15: 数据流标识参数 35

表 16: 函数概览 36

表 17: 接口状态 40

表 18: 设备信息介绍 58

表 19: 组网信息 63

表 20: 参考文档 69

表 21: 术语缩写 69

图片索引

图 1: 路由器功能图..... 10

图 2: QL_NF 整体框图 12

图 3: NF_INTF 状态变迁图 17

图 4: 拨号示意图 28

图 5: 数据拨号状态变迁图..... 30

图 6: ACL 工作示意图 34

图 7: 场景一 Linux 网络拓扑图..... 45

图 8: 场景一 QL_NF 网络拓扑图..... 46

图 9: 场景二 Linux 网络拓扑图..... 49

图 10: 场景二 QL_NF 网络拓扑图..... 51

图 11: 场景三 Linux 网络拓扑图 54

图 12: 场景三 QL_NF 网络拓扑图..... 56

图 13: EVB 环境 58

图 14: 测试组网 59

图 15: 场景四 Linux 网络拓扑图..... 64

图 16: 场景四 QL_NF 网络拓扑图..... 66

1 引言

移远通信 EC200A-CN(TA)模块支持 QuecOpen[®]方案。QuecOpen[®]是基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen 的详细信息，请参考文档 [1]。

本文档主要介绍移远通信 EC200A-CN(TA)模块的网络管理框架(QL_NF)，涉及数据拨号、LAN 管理、网络接口管理及数据转发策略管理等。

2 网络管理概念解释

本章引用常用路由的功能图，解释网络管理相关概念。路由器的简单配置和功能图如下。

表 1：路由器相关配置

WAN 口数量	1 个，以太网口
LAN 口数量	2 个，以太网口
WIFI	支持
防火墙	支持
管理方式	HTTP Server

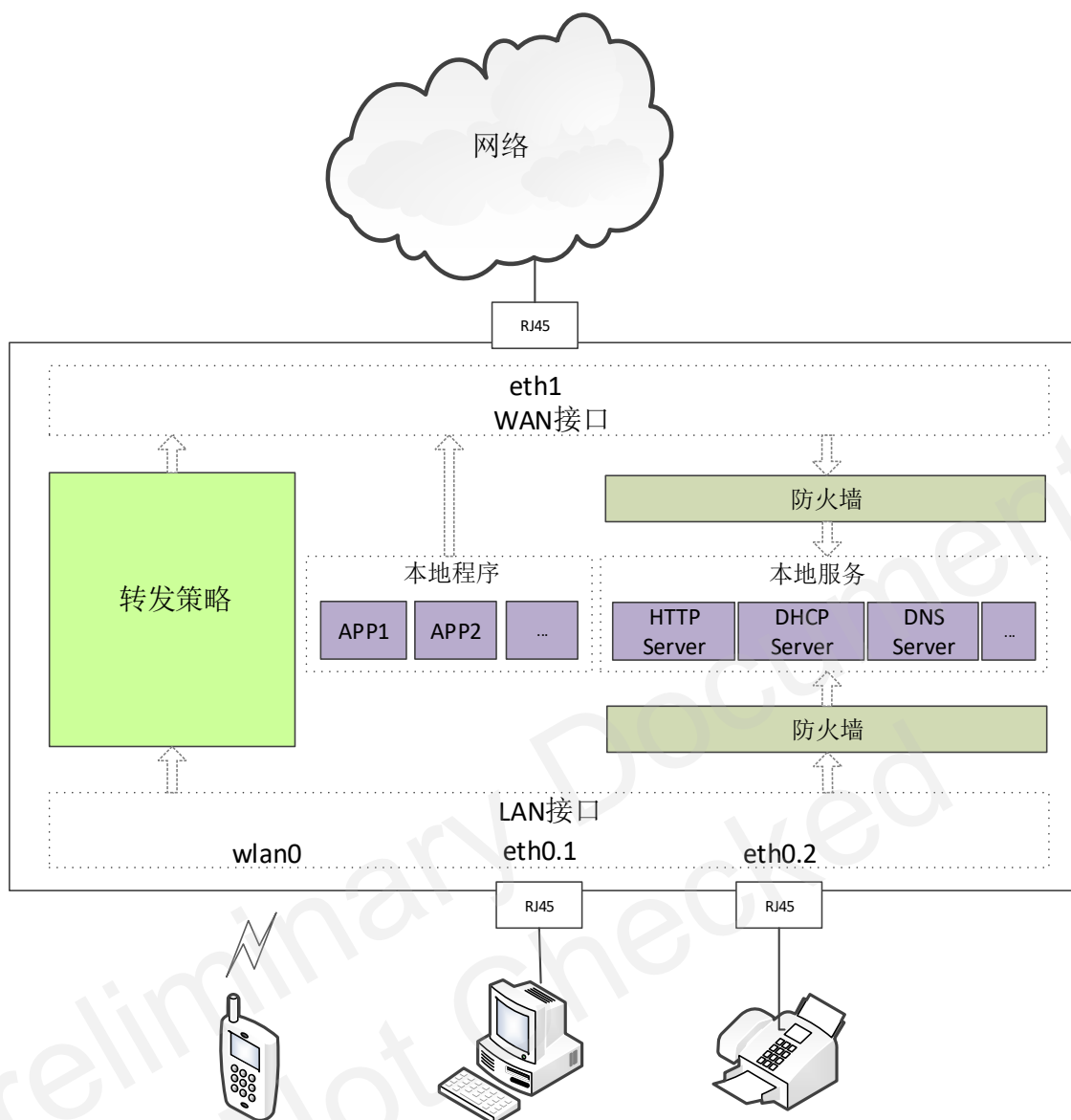


图 1：路由器功能图

- 网络接口（Network Interface）：Linux 系统的网卡名。如上图所示，以太网对应网络接口为 eth1、eth0.1 和 eth0.2；WIFI 对应的网络接口为 wlan0。
 - 1) WAN 接口（WAN Network Interface）：连接外部网络的 Linux 网络接口。如上图，通过以太网接入 Internet 网络，对应的 WAN 接口为 eth1。可以同时存在多个 WAN 接口。在移远通信模块上，WAN 接口还可以为拨号网络接口、USB 设备网络接口（ECM、RNDIS 和 NCM）和 WIFI Station 模式网络接口等。
 - 2) LAN 接口（LAN Network Interface）：接入 LAN 设备的网络接口。手机通过 Wi-Fi 接入路由器，PC 和打印机通过 ETH 接入路由器。对应的 LAN 口分别为 wlan0、eth0.1 和 eth0.2。在移远通信模块上，LAN 接口还可以为 USB 网络接口（ECM、RNDIS 和 NCM）等。
- LAN 设备（LAN Device）：通过 LAN 口，接入局域网的设备。如上图中的手机、PC 和打印机等。

- 本地服务 (Local Service): 在路由器运行的网络服务程序。例如上图中的 DHCP Server、DNS Server 和 HTTP Server。
- 本地程序 (Local Application): 在路由器运行且会访问网络的程序。如上图中的 APP1 和 APP2。
- DHCP Server: 为 LAN 设备提供 IP 地址管理服务。
- DNS Server: 为 LAN 设备提供域名解析服务。
- 防火墙 (Firewall): 阻止或者允许外部设备通过 WAN 口或 LAN 口访问本地服务。如上图, LAN 设备需要通过 DHCP Server 获取 IP 地址, 通过 DNS Server 解析域名, 通过 HTTP Server 管理路由器, 防火墙允许 LAN 设备访问 DHCP Server、DNS Server 和 HTTP Server, 禁止外部网络设备访问 DHCP Server 和 DNS Server, 同时出于安全考虑, 也禁止其 HTTP Server。
- 转发策略 (Forwarding Policy): LAN 设备通过路由器访问外部网络, 路由器内的一些网络配置策略, 包括路由处理和转发处理等。

3 QL_NF 介绍

传统的网络管理基于 Linux 的网络接口，实现路由和转发等策略。网络接口状态变化以及各种网络工具的配合使用，导致网络处理逻辑复杂而且难以维护。对于一些复杂应用场景，开发者需要了解各种网络协议、Linux 的网络技术（route、bridge 和 netfilter 等）和网络工具的使用（IP、iptables、ebtables 和 brctl 等）方可进行开发，这不但加长了开发周期，而且一些需要依靠经验解决的问题无法得到解决。

为解决这个问题，移远通信根据实际应用场景，把网络设备管理、转发策略等封装为一系列可描述的命令，开发者无需管理技术细节和了解路由和转发处理，只需了解第2章的相关概念，设置相应的规则，结合实际应用场景，即可完成规则的配置。

如上图所示，基于 Linux 网络设备接口，移远通信封装了一层网络接口（Quectel Network Interface），后面简称 NF_INTF，LAN 管理、WAN 管理、转发策略等均基于 NF_INTF。

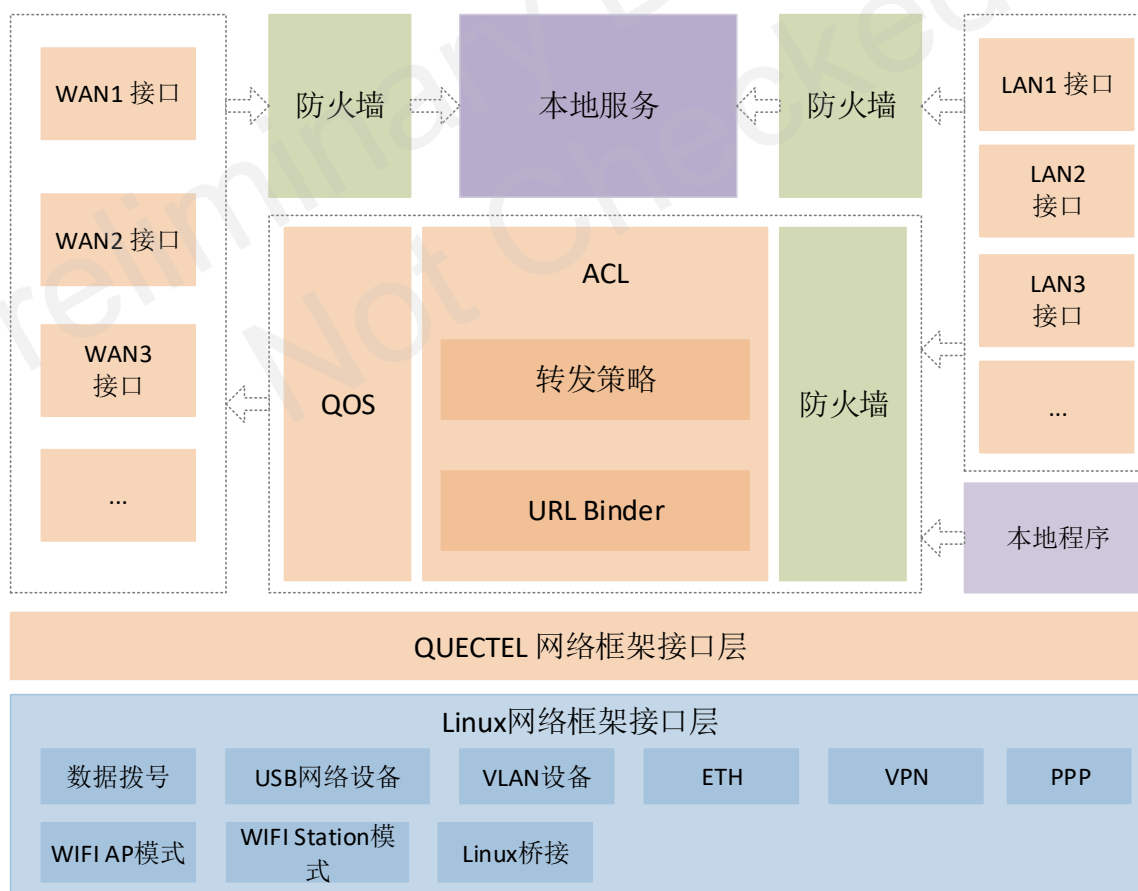


图 2: QL_NF 整体框图

3.1. QL_NF 规则配置

QL_NF 规则以可描述命令方式配置，当前支持普通格式配置。

- 命令格式

```
<action> <nf_target> <target_name> <param_list>
```

- 参数说明

<action> QL_NF 目标规则操作。支持如下操作：

add: 添加规则

del: 删除规则

set: 配置规则

get: 获取规则信息或配置

start: 使能规则

stop: 去使能规则

<nf_target> QL_NF 目标。支持如下目标：

interface: NF_INTF 管理

acl: 转发策略管理。URL 绑定管理

data_call: 数据拨号管理

dns_srv: DNS 服务管理

dhcp_srv: DHCP 服务管理

networks: 无线网络管理。包括无线网络注册，无线网络异常处理等。

qos: QOS 管理（待实现）

firewall: 防火墙管理（待实现）

<target_name> 由<nf_target>定义，通常为 QL_NF 目标实例的唯一标识。*all* 是特殊关键字，表示所有 QL_NF 目标。

<param_list> 参数列表。支持同时获取或配置多个参数，参数之间以空格分隔。设置参数时，参数名和参数值以 '=' 号连接，例如：param1 = value1 和 param2 = value2。

QL_NF 规则可以通过以下三种方式配置：

1. 预加载方式（推荐方式）。系统启动后，QL_NF 服务加载/etc/ql_nf_preload.conf 文件中的配置规则。
2. API 接口。头文件为 ql_nf.h，示例代码参见 sample/nf。
3. shell 命令：ql_cmd nf xxxx。建议在调试时使用此方式，调试完毕后，采用方式1 固化配置。

- 特殊命令

get all: 获取当前支持的所有 QL_NF 目标。

del all: 删除所有规则。

3.2. NF_INTF

3.2.1. NF_INTF 规则配置

- 命令格式

```
<action> interface <interface_name> <param_list>
```

- 参数说明

<action>	NF_INTF 操作。支持如下： <i>add</i> : 添加 NF_INTF 实例 <i>del</i> : 删除 NF_INTF 实例 <i>set</i> : 设置 NF_INTF 参数 <i>get</i> : 获取 NF_INTF 状态或者配置，返回配置参数或者状态参数 <i>start</i> : 开始 NF_INTF 寻址（寻址方式参见第 3.2.2 章） <i>stop</i> : 停止 NF_INTF 寻址
<interface_name>	NF_INTF 名称。单位：字节，最大长度 32 字节。NF_INTF 的唯一标识，其他 QL_NF 目标规则都使用<interface_name>引用对应的 NF_INTF。
<param_list>	参数列表

- 示例

添加 NF_INTF 规则。详见第 3.2.4 章。

```
add interface <interface_name> <key_params>
```

删除 NF_INTF 规则：

```
del interface <interface_name>
```

配置 NF_INTF 规则。<param_list>为配置参数列表，详见表 2。

```
set interface <interface_name> <param_list>
```

获取 NF_INTF 状态参数规则。状态参数详见表 3。若<interface_name>为 *all*，将获取所有 NF_INTF 的状态参数。

```
get interface <interface_name> class=info
```

获取 NF_INTF 配置参数。配置参数详见表 2。若<interface_name>为 *all*，将获取所有 NF_INTF 配置参数。

```
get interface <interface_name> class=config
```

● 参数

NF_INTF 有状态参数和配置参数两组参数。**set** 操作设置配置参数，**get** 操作获取配置参数和状态参数，参见下表。

表 2: NF_INTF 配置参数

参数	类型	描述
<i>proto</i>	字符串	寻址方式。支持的寻址方式如下： <i>none</i> 未设置寻址方式 <i>dhcp</i> 使用 DHCP 协议寻址 <i>static</i> 静态 IP 地址 <i>inherit</i> 继承网络接口的寻址方式，当前只有 <i>data_call</i> 类型会自动设置为该方式。
<i>mtu</i>	整数	MTU 值。范围：1~1518。
<i>mac</i>	MAC 地址格式字符串	MAC 地址
<i>ip</i>	IP 地址格式字符串	设置 IP 地址（ <i>static</i> 寻址方式使用）
<i>netmask</i>	IP 地址格式字符串	设置子网掩码（ <i>static</i> 寻址方式使用）
<i>gateway</i>	IP 地址格式字符串	设置网关（ <i>static</i> 寻址方式使用）
<i>dnsp</i>	IP 地址格式字符串	首选 DNS。注意：设置的首选 DNS 会覆盖 NF_INTF 使用其他寻址方式获取的 DNS。
<i>dnss</i>	IP 地址格式字符串	备选 DNS。注意：设置的备选 DNS 会覆盖 NF_INTF 使用其他寻址方式获取的 DNS。
<i>conn</i>	字符串	自动重连类型（DHCP 寻址方式使用，当寻址失败后，是否进行重试）。取值如下： <i>once</i> 不重试。NF_INTF 进入 <i>disconnected</i> 状态。 <i>auto</i> 自动重试
<i>vendor</i>	字符串	制造商名称（DHCP 寻址方式使用，DHCP OPTION 60 字段）。长度范围：1~32 字节。
<i>hostname</i>	字符串	主机名称（DHCP 寻址方式使用，DHCP OPTION 50 字段）。长度范围：1~32 字节。
<i>addif</i>	字符串	把传入的 NF_INTF 加入当前 <i>bridge</i> 类型的 NF_INTF 中。长度范围：1~32 字节。
<i>delif</i>	字符串	把传入的 NF_INTF 从当前 <i>bridge</i> 类型的 NF_INTF 中删除。长度范围：1~32 字节。

表 3: NF_INTF 状态参数

参数	类型	描述
<i>name</i>	字符串	NF_INTF 名称。长度范围：1~32 字节。
<i>status</i>	字符串	NF_INTF 状态。取值详见第 3.2.3 章。
<i>ifname</i>	字符串	对应 Linux 网络接口名称。长度范围：1~32 字节。
<i>ip</i>	IP 地址格式字符串	IP 地址
<i>netmask</i>	IP 地址格式字符串	子网掩码
<i>gateway</i>	IP 地址格式字符串	网关
<i>dnsp</i>	IP 地址格式字符串	首选 DNS
<i>dnss</i>	IP 地址格式字符串	备选 DNS

3.2.2. NF_INTF 寻址方式

NF_INTF 获取 IP 地址后，其他的 QL_NF 规则才能生效。NF_INTF 获取 IP 地址的过程，称为寻址。

当前支持寻址方式如下：

- *dhcp*: 使用 DHCP 协议获取 IP 地址。
- *static*: 静态 IP 地址（或称为手动 IP 地址）。若 *conn* 参数设置为 *auto* 模式，并且已经配置静态 IP 地址，NF_INTF 会立即进入 *connected* 状态。
- *inherit*: 继承 Linux 网络接口的寻址方式，目前只有 *data_call* 使用该寻址方式。

3.2.3. NF_INTF 状态

NF_INTF 支持 8 种状态，如下：

- *none*: 对应的 Linux 网络接口不存在。
- *unconfig*: 对应的 Linux 网络接口存在，但未配置寻址方式。
- *ready*: 已经配置寻址方式，可以使能寻址。
- *binder*: NF_INTF 绑定在桥接 NF_INTF 下，作为一个桥接端口。
- *connecting*: 正在寻址。

- *connected*: 寻址成功，获取到 IP 地址。
- *disconnected*: 寻址失败或者断开连接。

状态变迁图如下：

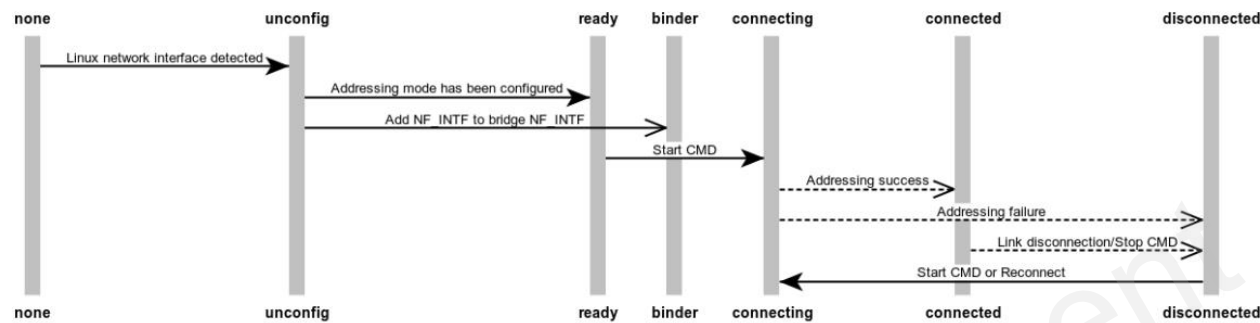


图 3：NF_INTF 状态变迁图

SDK 包中提供了 NF_INTF 状态变化上报的接口函数，参见 SDK 的 sample 程序 *sample/nf*。若用户应用程序需要监控网络状态变化，请参考第 3.10 章。

3.2.4. 创建 NF_INTF

- 命令格式

```
add interface <interface_name> type=<interface_type> <key_params>
```

- 参数说明

<interface_name> NF_INTF 名称。唯一标识 NF_INTF。
<interface_type> NF_INTF 类型。详见表 4。
<key_params> 关键参数。NF_INTF 通过关键参数建立和 Linux 网络接口的映射。具体关键参数描述，详见表 4 和表 5。

创建 NF_INTF 时，根据 NF_INTF 类型 (<interface_type>) 传入对应的关键参数 (<key_params>), NF 建立 NF_INTF 和 Linux 网络接口的映射关系。当前支持的 NF_INTF 类型和关键参数如下表。

表 4：NF_INTF 类型 (<interface_type>) 和关键参数 (<key_params>)

NF_INTF 类型	关键参数	描述
data_call	call_id	数据拨号接口。参考第 3.8 章。

<i>usb_dev</i>	<i>/</i>	USB 网络接口。自动识别 USB 网络接口名称（ <i>ecm0</i> 、 <i>usbnet0</i> 等）
<i>bridge</i>	<i>ifname</i>	桥接设备。需要传入网络接口名称（ <i>ifname</i> ），NF_INTF 自动创建桥接的网络接口。
<i>vdev</i>	<i>vlan</i> <i>master</i>	VLAN 接口。传入主 NF_INTF 名称（ <i>master</i> ）和 VLAN ID（ <i>vlan</i> ）。在主 NF_INTF 基础上创建一个 VLAN NF_INTF。
<i>com_dev</i>	<i>ifname</i>	通用接口。根据传入的网络接口名称（ <i>ifname</i> ），自动识别网络接口。
<i>local</i>	<i>/</i>	本地接口。特殊接口，表示所有本地服务和程序，系统自动创建，用户没有权限操作该接口。
<i>wifi_ap</i>	<i>index</i>	Wi-Fi AP 模式对应接口
<i>wifi_sta</i>	<i>/</i>	Wi-Fi STA 模式对应的接口

表 5: NF_INTF 关键参数 (<key_params>) 描述

关键参数	类型	描述
<i>call_id</i>	整数	数据拨号的唯一标识。范围：1~INT MAX。
<i>ifname</i>	字符串	Linux 网络接口名。范围：1~32 字节。
<i>vlan</i>	整数	VLAN ID。范围：1~4096。
<i>master</i>	字符串	主 NF_INTF 名称。范围：1~32 字节。

● 示例

创建各种类型的 NF_INTF 示例如下。

data_call 类型:

```
add interface intf_public type=data_call call_id=1
```

usb_dev 类型:

```
add interface intf_usb type=usb_dev
```

bridge 类型:

```
add interface intf_lan type=bridge ifname=br0
```

vdev 类型:

```
add interface intf_usb_300 type=vdev master=intf_usb vlan=300
```

com_dev 类型:

```
add interface intf_eth type=com_dev ifname=eth0
```

3.2.5. NF_INTF 配置示例

创建 *usb_dev* 类型的 NF_INTF，使用 DHCP 方式寻址，步骤如下。

步骤一 添加 *usb_dev* 类型的 NF_INTF。

```
add interface intf_usb type=usb_dev
```

步骤二 设置 NF_INTF 寻址方式和自动重连类型。

```
set interface intf_usb proto=dhcp conn=auto
```

步骤三 使能 NF_INTF，开始寻址。

```
start interface intf_usb
```

步骤四 查看 NF_INTF 状态。

```
get interface intf_usb class=info
```

3.3. VLAN 管理

QL_NF 支持基于 LAN 接口创建 VLAN 接口，只支持创建单层 VLAN 标签的 VLAN 接口。在实际运用中，通常使用 USB 网络接口和以太网接口创建对应的 VLAN 接口。可以使用 QL_NF 创建不多于 64 个 VLAN NF_INTF。

- 示例

首选需要创建 USB 网络接口以及以太网接口对应的 NF_INTF:

```
add interface intf_usb type=usb_dev
```

```
add interface intf_eth type=com_dev ifname=eth0
```

创建 VLAN ID 为 100、基于 USB 网络接口的 VLAN NF_INTF:

```
add interface intf_vdev1 type=vdev master=intf_usb vlan=100
```

创建 VLAN ID 为 200、基于 USB 网络接口的 VLAN NF_INTF:

```
add interface intf_vdev2 type=vdev master=intf_usb vlan=200
```

创建 VLAN ID 为 10、基于以太网接口的 VLAN NF_INTF:

```
add interface intf_vdev3 type=vdev master=intf_eth vlan=10
```

创建 VLAN ID 为 20、基于以太网接口的 VLAN NF_INTF:

```
add interface intf_vdev4 type=vdev master=intf_eth vlan=20
```

QL_NF 也支持设置 VLAN 接口发出数据报文中的 VLAN 优先级字段 (PCP)。在创建 VLAN NF_INTF 时, 使用 *vlan_pri* 参数传入 VLAN 优先级值。使用示例如下:

创建 VLAN ID 为 300、输出数据报文的 VLAN 优先级为 4、基于 USB 网络接口的 VLAN NF_INTF:

```
add interface intf_vdev5 type=vdev master=intf_usb vlan=300 vlan_pri=4
```

创建 VLAN ID 为 30、输出数据报文的 VLAN 优先级为 5、基于以太网接口的 VLAN NF_INTF:

```
add interface intf_vdev6 type=vdev master=intf_eth vlan=30 vlan_pri=5
```

3.4. 桥接管理

桥接是一种将采用不同接入方式接入模块的设备接入同一个局域网的方法。在 QL_NF 中, 任意类型的 NF_INTF 都可以加入桥接 NF_INTF, 常用的是把 LAN NF_INTF 加入桥接 NF_INTF。常用的 LAN 接口有: USB 网络接口、以太网接口、Wi-Fi 接口、基于 USB 网络接口以及以太网接口的 VLAN 接口。

桥接 NF_INTF 的使用方法为: 创建 LAN NF_INTF; 创建桥接 NF_INTF; 把 LAN NF_INTF 加入桥接 NF_INTF。

● 示例

把 USB 网络接口、以太网接口和基于以太网接口的 VLAN ID 为 200 的 VLAN 接口，加入一个桥接接口。

步骤一 创建 USB 网络接口、以太网接口以及 VLAN 接口对应的 NF_INTF:

```
add interface intf_usb type=usb_dev
add interface intf_eth type=com_dev ifname=eth0
add interface intf_eth_v200 type=vdev master=intf_eth vlan=200
```

步骤二 创建桥接 NF_INTF:

```
add interface intf_br type=bridge ifname=bridge1
```

步骤三 把相应的 LAN 接口加入桥接接口:

```
set interface intf_br addif=intf_usb
set interface intf_br addif=intf_eth
set interface intf_br addif= intf_eth_v200
```

● 参数

桥接接口同时支持额外的配置参数，详见表 6。

表 6：桥接接口扩展配置参数

参数	类型	描述
noarp	整数	打开或禁止桥接接口的 ARP 功能。若禁止 ARP 功能，则使用静态 ARP。
		0 打开（默认值） 1 禁止
isolate	整数	打开或关闭桥接接口下 LAN 接口之间的数据隔离。
		0 关闭（默认值） 1 打开

3.5. Wi-Fi 管理

QL_NF 支持管理使用 QuecOpen Wi-Fi API 创建的 Wi-Fi 接口。QuecOpen Wi-Fi API 支持创建 1~2 个 AP 模式的网络接口以及一个 STA 模式的网络接口，详情请参考文档 [2]。

创建 Wi-Fi 相关的 NF_INTF 以及相应的转发规则不需要 Wi-Fi 网络接口已经存在，QL_NF 可以自动识

别是否创建和删除了 Wi-Fi 网络接口及其状态变化，并动态更改相应 NF_INTF 的状态。在开发过程中，可以将 Wi-Fi 相关的规则预设置在 `/etc/ql_nf_preload.conf` 文件中，然后在应用程序中，使用 Wi-Fi API 管理 Wi-Fi。

创建 Wi-Fi 的 NF_INTF 示例如下：

- 示例

创建 Wi-Fi AP0 的 NF_INTF：

```
add interface intf_wifi_ap0 type=wifi_ap index=0
```

创建 Wi-Fi AP1 的 NF_INTF：

```
add interface intf_wifi_ap1 type=wifi_ap index=1
```

创建 Wi-Fi STA 模式的 NF_INTF：

```
add interface intf_wifi_sta type=wifi_sta
```

3.6. LAN 管理

LAN 管理包括 DHCP 服务、DNS 服务和 LAN 设备的管理。DHCP 服务为 LAN 设备分配 IP 地址。DNS 服务为 LAN 设备提供域名解析服务。

LAN 设备可以通过 ETH、USB 和 Wi-Fi 等方式接入同一个局域网。接入局域网的过程中需要使用桥接方式，把 ETH、USB 和 Wi-Fi 对应的 NF_INTF 加到桥接 NF_INTF 下，然后在桥接 NF_INTF 上管理 DHCP 服务。`/etc/ql_nf_preload.conf` 文件中的预置规则即是创建 USB、ETH 和桥接类型的 NF_INTF，并把 USB 和 ETH 对应的 NF_INTF 加入到桥接 NF_INTF。

```
ql_cmd add interface intf_usb type=usb_dev
ql_cmd add interface intf_lan type=bridge ifname=br0 proto=static conn=auto
ql_cmd set interface intf_lan ip=192.168.225.1 netmask=255.255.255.0
ql_cmd set interface intf_lan addif=intf_usb
```

3.6.1. DHCP 服务管理

QL_NF 使用 `dhcp_srv` 管理 DHCP 服务，一个 DHCP 服务和一个 LAN NF_INTF 绑定，支持同时启动多个 DHCP 服务。

● 命令格式

```
<action> dhcp_srv <dhcp_srv_name> <param_list>
```

● 参数解释

<action>	DHCP 服务操作。支持操作如下： <i>add</i> : 添加 DHCP 服务实例 <i>del</i> : 删除 DHCP 服务实例 <i>set</i> : 配置 DHCP 服务 <i>get</i> : 获取 DHCP 服务状态或者配置。返回配置参数（详见表 7）或者状态参数（详见表 8）。 <i>start</i> : 开启 DHCP 服务 <i>stop</i> : 停止 DHCP 服务
<dhcp_srv_name>	DHCP 服务名称。最大长度 32 字节，DHCP 服务的唯一标识。
<param_list>	参数列表

● 示例

添加 DHCP 服务：

```
add dhcp_srv <dhcp_srv_name>
```

配置 DHCP 服务参数。**<param_list>**为配置参数列表，详见表 7。

```
set dhcp_srv <dhcp_srv_name> <param_list>
```

开启 DHCP 服务：

```
start dhcp_srv <dhcp_srv_name>
```

停止 DHCP 服务：

```
stop dhcp_srv <dhcp_srv_name>
```

获取 DHCP 服务状态参数。返回状态参数列表，详见表 8。

```
get dhcp_srv <dhcp_srv_name> class=info
```

获取 DHCP 服务配置参数，返回配置参数列表，详见表 7。

```
get dhcp_srv <dhcp_srv_name> class=config
```

表 7：配置参数

参数	类型	描述
<i>intf</i>	字符串	绑定的 LAN NF_INTF。长度范围：1~32 字节。
<i>ip_start</i>	IP 地址格式字符串	DHCP 地址池起始地址
<i>ip_end</i>	IP 地址格式字符串	DHCP 地址池结束地址
<i>netmask</i>	IP 地址格式字符串	子网掩码
<i>gateway</i>	IP 地址格式字符串	网关地址。注意，网关地址必须和绑定的 NF_INTF 的 IP 地址相同。
<i>dnsp</i>	IP 地址格式字符串	首选 DNS。常设置为网关地址。
<i>dnss</i>	IP 地址格式字符串	备选 DNS。常设置为空。
<i>lease_time</i>	整数	DHCP 地址租期。单位：小时，范围：1~INT MAX。 建议设置为 12 小时以上，

表 8：状态参数

参数	类型	描述
<i>name</i>	字符串	DHCP 服务名称。长度范围：1~32 字节。
<i>start</i>	整数	服务开启状态。
		0 未开启
		1 已开启
<i>status</i>	字符串	DHCP 服务状态。
		<i>none</i> 未启动
		<i>active</i> 正在运行
		<i>error</i> 发生内部错误

3.6.1.1. DHCP 配置示例

通过 */etc/ql_nf_preload.conf* 文件中的预置命令创建桥接 NF_INTF，并启动 192.168.225.0 网段的 DHCP 服务。

```
add interface intf_usb type=usb_dev
add interface intf_lan type=bridge ifname=br0
set interface intf_lan proto=static conn=auto ip=192.168.225.1 netmask=255.255.255.0
set interface intf_lan addif=intf_usb
add dhcp_srv dhcp_lan intf=intf_lan
set      dhcp_srv      dhcp_lan      ip_start=192.168.225.100      ip_end=192.168.225.200
```

```
netmask=255.255.255.0 gateway=192.168.225.1 dnsp=192.168.225.1 lease_time=86400
start dhcp_srv dhcp_lan
```

3.6.2. DNS 服务管理

QL_NF 使用 dns_srv 管理 DNS 服务，系统只能启动一个 DNS 服务。

- 命令格式

```
<action> dns_srv <dns_srv_name>
```

- 参数

<action> DNS 服务操作。支持类型如下：
add: 添加 DNS 服务实例
del: 删除 DNS 服务实例
get: 获取 DNS 服务状态，返回状态参数（详见表 9）。
start: 开启 DNS 服务
stop: 停止 DNS 服务

<dns_srv_name> DNS 服务名称。最大长度 32 字节，DNS 服务的唯一标识。

- 示例

添加 DNS 服务实例：

```
ql_cmd nf add dns_srv <dns_srv_name>
```

删除 DNS 服务示例：

```
ql_cmd nf del dns_srv <dns _srv_name>
```

开启 DNS 服务示例：

```
ql_cmd nf start dns_srv <dns _srv_name>
```

停止 DNS 服务示例：

```
ql_cmd nf stop dns_srv <dns _srv_name>
```

获取 DNS 服务状态示例。返回 DNS 状态参数，详见表 9。

```
ql_cmd nf get dns_srv <dns _srv_name>
```

表 9：DNS 服务状态参数

参数	类型	描述
<i>name</i>	字符串	DNS 服务名称。长度范围：1~32 字节。
<i>start</i>	整数	服务是否开启。 0 未开启 1 已开启
<i>status</i>	字符串	DNS 服务状态。 <i>none</i> 服务未启动 <i>active</i> 服务正在运行 <i>error</i> 发生内部错误

3.6.3. LAN 管理示例

LAN 设备通过 USB 和 ETH 接入到模块创建的局域网，局域网网段为 192.168.225.0/24。LAN 设备可以通过 DHCP 方式获取 IP 地址，并能 PING 通网关。对应的 QL_NF 命令示例如下。

```
add interface intf_usb type=usb_dev
add interface intf_lan type=bridge ifname=br0
set interface intf_lan proto=static conn=auto ip=192.168.225.1 netmask=255.255.255.0
set interface intf_lan addif=intf_usb
add dhcp_srv dhcp_lan intf=intf_lan
set      dhcp_srv      dhcp_lan      ip_start=192.168.225.100      ip_end=192.168.225.200
netmask=255.255.255.0 gateway=192.168.225.1 dnsp=192.168.225.1 lease_time=86400
start dhcp_srv dhcp_lan
add dns_srv dns_lan
start dns_srv dns_lan
```

3.6.4. LAN 主机管理

QL_NF 支持查询接入 QL_NF 管理的局域网的主机。

● 命令格式

```
get lanhost all
```

查询命令将返回 LAN 主机信息，如下表所示。

表 10: LAN 主机信息

参数	类型	描述
<i>ip</i>	IP 地址格式字符串	LAN 主机的 IP 地址
<i>mac</i>	MAC 地址格式字符串	LAN 主机的 MAC 地址
<i>name</i>	字符串	LAN 主机名称。根据 DHCP 请求报文中的字段获取，若主机使用静态 IP 地址方式或者 DHCP 请求报文中没有包含名称信息，则该字段为空。
<i>intf</i>	字符串	LAN 主机接入的局域网对应的 NF_INTF
<i>ifname</i>	字符串	LAN 主机接入的 Linux 接口名称
<i>uptime</i>	整数	在线时间。单位秒。

SDK 中同时提供 API 获取 LAN 主机信息，详情请参考头文件 `ql-sysroots/usr/include/ql-sdk/ql_lanhost.h`。

3.7. 静态 ARP 规则管理

QL_NF 支持创建、删除和查询静态 ARP 规则，命令格式如下。

● 命令格式

```
<action> interface <arp_rule_name> <param_list>
```

● 参数说明

<action> ARP 规则操作，支持如下：
add: 添加静态 ARP 规则
del: 删除静态 ARP 规则
get: 获取所有静态 ARP 规则

<arp_rule_name> ARP 规则名称。最大长度 32 字节，ARP 规则的唯一标识。

<param_list> 参数列表。详见表 11。

表 11: ARP 规则参数列表

参数	类型	描述
<i>intf</i>	字符串	ARP 规则基于的 NF_INTF

ip	IP 地址格式字符串	IP 地址
mac	MAC 地址格式字符串	MAC 地址

3.8. 数据拨号

3.8.1. 拨号介绍

无线数据业务的建立过程，即称为数据拨号（Data Call）。设备携带 APN 信息向运营商网络侧发起数据拨号，运营商网络侧根据 APN 信息分配 IP 地址，并建立数据链路到相应网络，具体流程如下图。

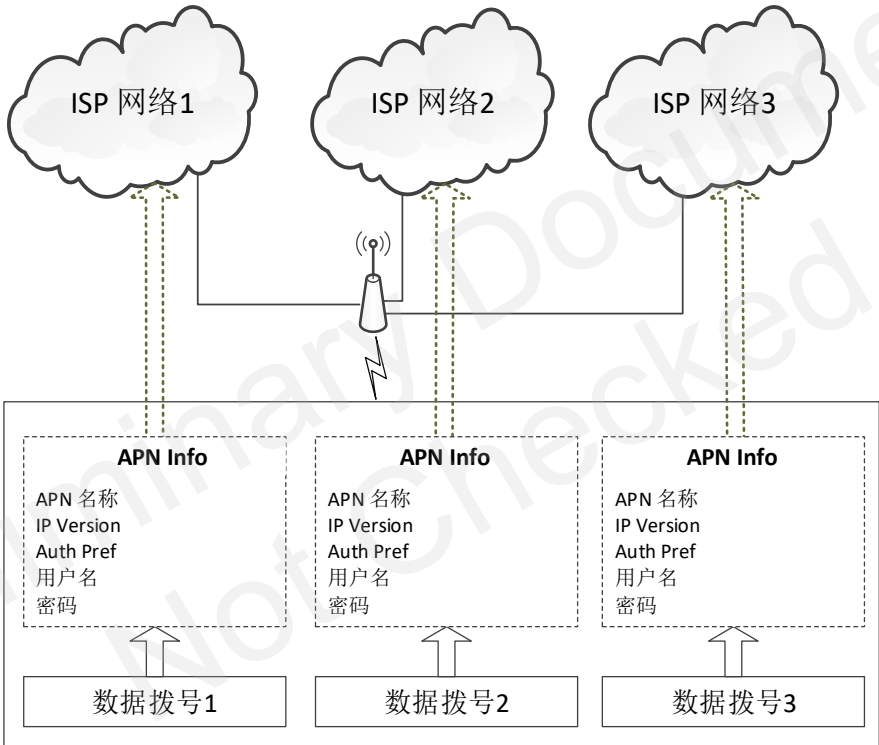


图 4：拨号示意图

运营商会签约时为 SIM 卡分配一个或多个 APN 信息，常见的是一路公网 APN 信息以及一路或者多路私网 APN 信息。模块携带 APN 信息进行数据拨号，运营商网络侧根据 APN 信息分配公网或者私网的 IP 地址，参见上图。

数据拨号成功的前提是模块已注册到运营商网络。LTE 网络注册过程中，需要建立默认承载。和数据拨号类似，建立默认承载也需要传入 APN 信息，若传入的 APN 信息错误，可能导致网络注册失败。运营商分配的 APN 信息中，有一路 APN 信息可以用来建立默认承载，详情请咨询运营商。

3.8.2. 重连机制

拨号失败或者拨号断开时，可选择是否进入自动重连模式。模块支持以下四种自动重连模式：

● **disable**

不自动重连，用户可以自己实现自动重连机制。

● **normal**

根据固定时间间隔进行重连，建议设置时间间隔 20 秒以上。频繁拨号可能会被运营商加入黑名单。

● **mode1**

以时间退避模式进行重连，例如时间间隔列表为 T1、T2、T3...Tn，重连的时间间隔为 T1、T2、T3...Tn、Tn、Tn....Tn 直至拨号成功。若需再次重连，则间隔时间从 T1 开始。

● **mode2**

循环时间退避模式进行重连，例如时间间隔列表为 T1、T2、T3...Tn，重连的时间间隔为 T1、T2、T3...Tn、T1、T2、T3...Tn....直至拨号成功。若需再次重连，则间隔时间从 T1 开始。

3.8.3. 设置默认承载 APN

● **命令格式**

```
set networks attach_apn <param_list>
```

● **参数说明**

<param_list> APN 参数列表。详见表 12。

表 12：APN 参数列表

参数	类型	描述
apn_name	字符串	APN 名称。运营商根据此参数分配 IP 地址信息。长度范围：1~150 字节。
ip_ver	整数	IP 类型。
		4 IPv4
		6 IPv6
auth_pref	字符串	46 IPv4v6
		认证方式。部分运营商网络会开启认证，需要传入认证方式，如无特殊指明，可不设置该参数或者设置为 none。

3.8.5. 数据拨号命令

- 命令格式

```
<action> data_call <call_id> <param_list>
```

- 参数说明

<action> 数据拨号实例操作。支持操作类型如下：

add: 添加一路数据拨号实例

del: 删除数据拨号实例

set: 配置数据拨号参数

get: 获取数据拨号状态参数或者配置参数

start: 开始数据拨号

stop: 停止数据拨号

<call_id> 数据拨号的唯一标识。整型，取值范围为 1~INT MAX。

<param_list> 配置参数列表

- 示例

创建数据拨号实例：

```
ql_cmd nf add data_call <call_id>
```

删除数据拨号实例：

```
ql_cmd nf del data_call <call_id>
```

设置数据拨号配置参数：

```
ql_cmd nf set data_call <call_id> <param_list>
```

开始数据拨号：

```
ql_cmd nf start data_call <call_id>
```

停止数据拨号：

```
ql_cmd nf stop data_call <call_id>
```

获取数据拨号状态。返回状态参数列表详见表 14。若<call_id>为 all，表示获取所有数据拨号状态。

```
ql_cmd nf get data_call <call_id> class=info
```

获取拨号配置参数。返回配置参数列表详见表 13。若<call_id>为 *all*，表示获取所有数据拨号配置参数。

```
ql_cmd nf get data_call <call_id> class=config
```

表 13：数据拨号配置参数

参数	类型	描述
<i>call_id</i>	整数	用户指定，唯一标识数据拨号实例。范围：1~INT MAX。
<i>apn_name</i>	字符串	APN 名称。运营商网络根据此参数分配 IP 地址信息，长度范围：1~50 字节。
<i>ip_ver</i>	整数	IP 类型。 4 IPv4 6 IPv6 46 IPv4v6
<i>auth_pref</i>	字符串	认证方式。部分运营商网络会开启认证，需要传入认证方式，若无特殊指明，可不设置该参数或者设置为 <i>none</i> 。 <i>none</i> 无认证（默认值） <i>pap</i> PAP 认证 <i>chap</i> CHAP 认证 <i>pap_chap</i> PAP 和 CHAP 认证
<i>username</i>	字符串	用户名。长度范围：1~150 字节。
<i>password</i>	字符串	密码。长度范围：1~150 字节。
<i>reconn_mode</i>	字符串	自动重连模式。详见第 3.8.2 章。
<i>reconn_interval</i>	整数数组	自动重连时间间隔列表。单位：秒，数组最大个数为 20。
<i>sim_id</i>	整数	若模块支持 DSDA，则使用该参数指定(U)SIM 卡槽； 若模块不支持 DSDA，无需设置该参数。 1 卡槽 1 2 卡槽 2

表 14：状态参数

参数	类型	描述
<i>status</i>	字符串	拨号状态。详见第 3.8.4 章。

<i>ipaddr</i>	IP 地址格式字符串	IP 地址
<i>netmask</i>	IP 地址格式字符串	子网掩码
<i>gateway</i>	IP 地址格式字符串	网关
<i>dnsp</i>	IP 地址格式字符串	首选 DNS
<i>dnss</i>	IP 地址格式字符串	备选 DNS
<i>last_ecode</i> (暂不支持)	整数	上次拨号失败或者断开的内部错误码。错误码详见头文件 <i>ql-sysroots/usr/include/ril.h</i> 中的 <i>RIL_LastCallFailCause</i> 结构体

3.8.6. 拨号示例

若用户的(U)SIM卡和运营商签约了两路APN, 公网APN名称为apnpublic, 私网APN名称为apnprivate, 其中公网APN可以建立默认承载。

```
ql_cmd nf set networks attach_apn apn_name=apnpublic
ql_cmd nf add data_call 1
ql_cmd nf set data_call 1 apn_name=apnpublic
ql_cmd nf start data_call 1
ql_cmd nf add data_call 2
ql_cmd nf set data_call 2 apn_name=apnprivate
ql_cmd nf start data_call 2
```

3.8.7. 注意事项

用户可通过API函数管理数据拨号、使用QL_NF管理转发规则。有关API函数的详情请参考文档 [3]。

数据拨号API函数使用call_id唯一标识数据拨号实例。在QL_NF中创建数据拨号NF_INTF, 也是使用call_id关联数据拨号的实例。创建NF_INTF时, 不需要数据拨号实例存在; 创建数据拨号实例后, 会自动同步数据拨号NF_INTF的状态。在开发过程中, 可以将数据拨号的NF_INTF以及相关的转发规则预设置在/etc/ql_nf_preload.conf配置文件中, 然后在应用程序中使用数据拨号的API函数管理数据拨号。

3.9. 转发策略

3.9.1. 转发策略介绍

转发策略决定LAN接口和WAN接口之间的选路、本地程序和WAN接口之间的选路。QL_NF使用ACL管理转发策略。

ACL表示访问控制列表(Access Control Lists), 用于设置LAN接口和WAN接口之间的选路、本地

程序和 WAN 接口之间的选路。用户只需指定数据源（LAN NF_INTF 或者本地程序）、数据类型以及数据目的（WAN NF_INTF），即可完成规则的配置，无需对 Linux 网络规则进行配置。

为了管理本地程序的网络访问，QL_NF 使用 intf_local NF_INTF 表示所有模块上运行的本地程序。工作示意图如下：

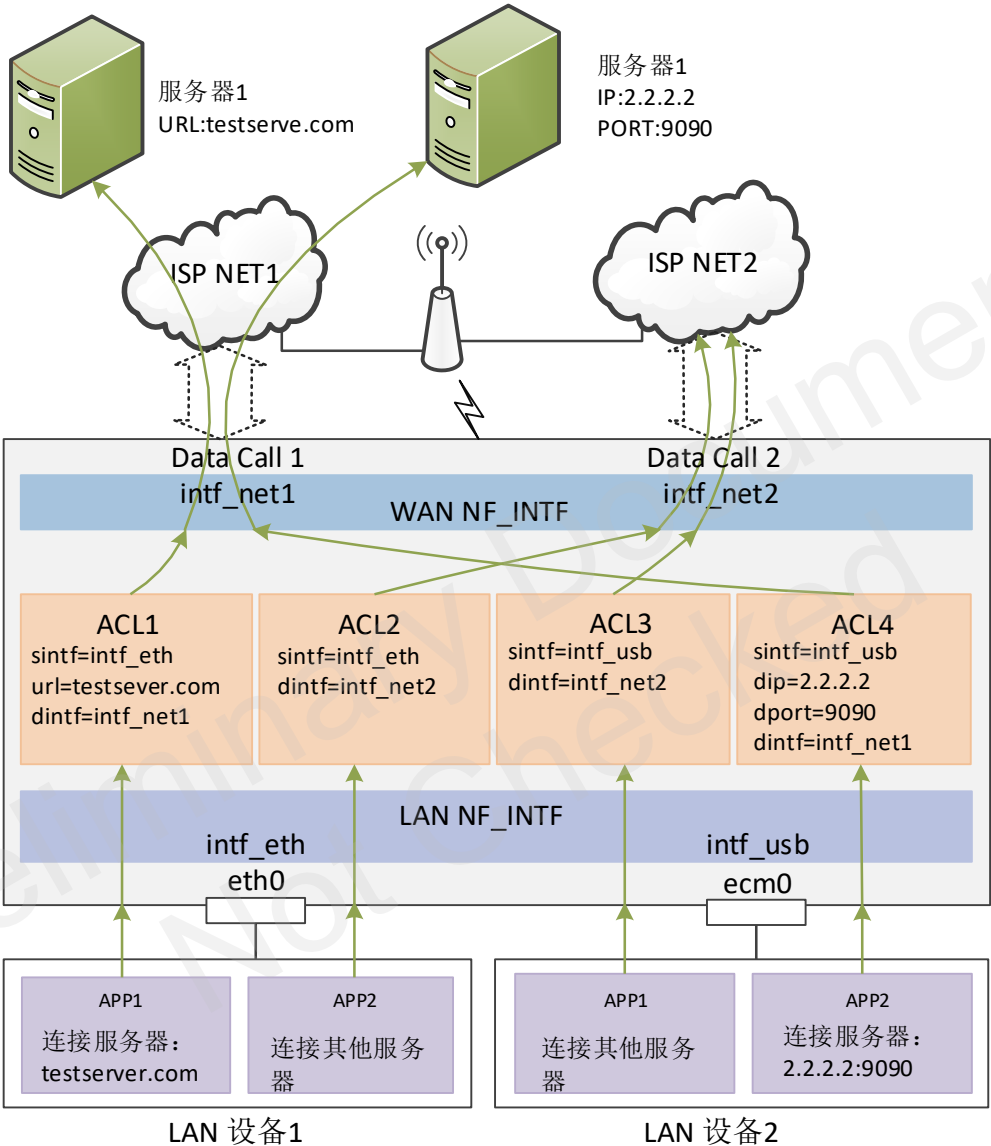


图 6：ACL 工作示意图

3.9.2. ACL 命令

- 命令格式

```
<action> acl <acl_name> sintf=<source_interface_name> <data_flow_type_params>  
dintf=<destination_interface_name>
```

● 参数说明

<action>	ACL 规则操作。支持操作类型如下： <i>add</i> : 添加 ACL 规则 <i>del</i> : 删除 ACL 规则 <i>get</i> : 获取 ACL 配置
<acl_name>	ACL 规则名称。ACL 规则的唯一标识，字符串类型，1~32 字节长度。
<source_interface_name>	LAN NF_INTF 名称。可设置为 <i>intf_local</i> ，表示所有本地服务和本地程序。
<data_flow_type_params>	数据流标识参数列表。详见表 15。
<destination_interface_name>	WAN NF_INTF 名称。数据转发的出口。

● 示例

添加 ACL 规则：

```
add acl <acl_name> sintf=<source_interface_name> <data_flow_type_params> dintf=<destination_interface_name>
```

删除 ACL 规则：

```
del acl <acl_name>
```

查看所有 ACL 规则：

```
get acl all
```

删除所有 ACL 规则：

```
del acl all
```

表 15：数据流标识参数

参数	类型	描述
<i>proto</i>	字符串	数据流类型。本地程序不支持该参数。
		<i>all</i> 所有类型。包括 TCP 和 UDP。
		<i>tcp</i> TCP 流
		<i>udp</i> UDP 流
<i>url</i>	字符串	URL 绑定。绑定 URL 到对应的 WAN 接口，长度范围：1~128 字节。
		此参数不能同<proto>、<sport>、<dip>或<dport>共用。

smac	MAC 地址格式字符串	数据流源 MAC 地址。即 LAN 设备 MAC 地址，本地程序不支持该参数。
sip	IP 地址格式字符串	数据流源 IP 地址。即 LAN 设备 IP 地址，本地程序不支持该参数。
sport	整数	数据流源端口。本地程序不支持该参数。范围：1~65535。
dip	IP 地址格式字符串	数据流目的 IP 地址
dport	整数	数据流目的端口。本地程序不支持该参数。范围：1~65535。

3.10. QL_NF API

3.10.1. 头文件

QL_NF API 头文件为 *ql_nf.h*，部分公用结构体定义在头文件 *ql_net_common.h*，位于 SDK 包的 *ql-sysroots/usr/include/ql-sdk* 目录下。若无特别说明，本章所涉及头文件均在该目录下。

3.10.2. 函数概览

表 16：函数概览

函数	说明
<i>ql_nf_init()</i>	初始化 QL_NF 服务
<i>ql_nf_cmd()</i>	发送普通格式的 QL_NF 命令，并等待响应
<i>ql_nf_set_interface_status_ind_cb()</i>	设置监控 NF_INTF 状态变化回调函数
<i>ql_nf_set_service_error_cb()</i>	设置服务程序异常通知的回调函数
<i>ql_nf_deinit()</i>	注销 QL_NF 服务

备注

若无特别说明，所有 QL_NF API 均不支持并发调用。

3.10.3. 函数详解

3.10.3.1. ql_nf_init

该函数用于初始化 QL_NF 服务。

- 函数原型

```
int ql_nf_init(void);
```

- 参数

无

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_ARG	无效参数
QL_ERR_UNKNOWN	未知错误
QL_ERR_SERVICE_NOT_READY	QL_NF 服务准备就绪，返回此错误码时，建议重试。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

使用其他 QL_NF API 前，必须调用本函数初始化 QL_NF 服务。

3.10.3.2. ql_nf_cmd

该函数用于发送普通格式的 QL_NF 命令，并等待响应。

- 函数原型

```
int ql_nf_cmd(const char *cmd, char *resp_buf, int resp_len);
```

- 参数

cmd:

[In] QL_NF 普通格式命令。详见第 3.1 章。

resp_buf:

[Out] 存放响应的数据缓冲区。QL_NF 命令的响应为字符串格式。

resp_len:

[In] *resp_buf* 缓冲区大小。单位：字节。

用户根据自己的命令需求设置响应缓冲区大小。例如发送获取单个 NF_INTF 信息的命令，缓冲区可设置为 256 字节大小；若获取所有 NF_INTF 信息的命令，建议缓冲区大小为 2048 字节以上。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_ARG	无效参数
QL_ERR_UNKNOWN	未知错误
QL_ERR_SERVICE_NOT_READY	QL_NF 服务准备就绪。返回此错误码时，建议重试。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.10.3.3. ql_nf_set_interface_status_ind_cb

该函数为设置监控 NF_INTF 状态变化的回调函数。

- 函数原型

```
int ql_nf_set_interface_status_ind_cb(ql_nf_interface_status_ind_cb_f cb);
```

- 参数

cb:

[In] 监控 NF_INTF 状态变化的回调函数。详见第 3.10.3.3.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	未初始化
QL_ERR_SERVICE_NOT_READY	QL_NF 服务准备就绪，返回此错误码时，建议重试。
QL_ERR_INVALID_ARG	无效参数
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.10.3.3.1.ql_nf_interface_status_ind_cb_f

该回调函数监控 NF_INTF 状态变化。

- 函数原型

```
typedef void (*ql_nf_interface_status_ind_cb_f)(ql_nf_interface_status_t *p_msg);
```

● 参数

p_msg:
[Out] NF_INTF 状态结构体。详见第3.10.3.3.2 章。

3.10.3.3.2.ql_nf_interface_status_t

NF_INTF 状态结构体定义如下：

```
typedef struct {
    char name[QL_NET_MAX_NAME_LEN];
    char ifname[QL_NET_MAX_NAME_LEN];
    char pre_status[16];
    char status[16];
    uint8_t has_addr;
    ql_net_addr_t addr;
    uint8_t has_addr6;
    ql_net_addr6_t addr6;
} ql_nf_interface_status_t;
```

● 参数

类型	参数	描述
char	<i>name</i>	NF 中的接口名称
char	<i>ifname</i>	网口名称
char	<i>pre_status</i>	接口先前状态。详见表 17。
char	<i>status</i>	接口当前状态。详见表 17。
uint8_t	<i>has_addr</i>	有无 IPv4 地址。 0 无 IPv4 地址 非 0 有 IPv4 地址
ql_net_addr_t	<i>addr</i>	IPv4 地址。详见第 3.10.3.3.3 章。
uint8_t	<i>has_addr6</i>	有无 IPv6 地址。 0 无 IPv4 地址 非 0 有 IPv4 地址
ql_net_addr6_t	<i>addr6</i>	IPv6 地址。详见第 3.10.3.3.4 章。

表 17：接口状态

接口状态	描述
NF_INTF_STATUS_NONE	初始状态或空闲状态
NF_INTF_STATUS_UNCONFIG	未配置
NF_INTF_STATUS_READY	就绪状态
NF_INTF_STATUS_CONNECTING	连接中
NF_INTF_STATUS_CONNECTED	已连接
NF_INTF_STATUS_DISCONNECTED	断开连接
NF_INTF_STATUS_ERROR	出错
NF_INTF_STATUS_BINDER	绑定到其他接口

3.10.3.3.3.ql_net_addr_t

IPv4 地址结构体定义如下：

```
typedef struct {
    char addr[QL_NET_MAX_ADDR_LEN];
    char netmask[QL_NET_MAX_ADDR_LEN];
    uint8_t subnet_bits;
    char gateway[QL_NET_MAX_ADDR_LEN];
    char dnsp[QL_NET_MAX_ADDR_LEN];
    char dnss[QL_NET_MAX_ADDR_LEN];
} ql_net_addr_t;
```

● 参数

类型	参数	描述
char	addr	Ipv4 地址。类型：字符串。
char	netmask	子网掩码
uint8_t	subnet_bits	网络号位数
char	gaterway	网关
char	dnsp	首 DNS
char	dnss	副 DNS

3.10.3.3.4.ql_net_addr6_t

IPv6 地址结构体定义如下：

```
typedef struct
{
    char addr[QL_NET_MAX_ADDR_LEN];
    char prefix[QL_NET_MAX_ADDR_LEN];
    int32_t prefix_bits;
    char gateway[QL_NET_MAX_ADDR_LEN];
    char dnsp[QL_NET_MAX_ADDR_LEN];
    char dnss[QL_NET_MAX_ADDR_LEN];
} ql_net_addr6_t;
```

● 参数

类型	参数	描述
char	addr	Ipv6 地址。类型：字符串。
char	prefix	Ipv6 前缀
uint8_t	prefix_bits	前缀位数
char	gaterway	网关
char	dnsp	首 DNS
char	dnss	副 DNS

3.10.3.4. ql_nf_set_service_error_cb

该函数用于设置通知服务程序异常的回调函数。

● 函数原型

```
int ql_nf_set_service_error_cb(ql_nf_service_error_cb_f cb) ;
```

● 参数

cb:
[In] 服务程序异常通知回调函数。详见第3.10.3.4.1 章。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

3.10.3.4.1.ql_nf_service_error_cb_f

该回调函数通知服务程序异常。

- 函数原型

```
typedef void (*ql_nf_service_error_cb_f)(int error);
```

- 参数

error:

[Out] 错误原因。类型：整数。用户自定义。

3.10.3.5. ql_nf_deinit

该函数用于注销 QL_NF 服务。

- 函数原型

```
int ql_nf_deinit(void);
```

- 参数

无

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

3.11. 注意事项

3.11.1. MAC 地址分配

每个 Linux 网络接口都有一个 MAC 地址，MAC 地址长度为 48 位（6 个字节），通常表示为 12 个 16 进制数，每 2 个 16 进制数之间用冒号隔开，如 08:22:22:22:22:22。

MAC 地址最高字节的第二个比特位表示该地址为全局地址还是本地地址。如果为 1，表示是本地 MAC

地址；如果为 0，表示是全局 MAC 地址。比如 08:22:22:22:22:22 是全局地址，02:22:22:22:22:22 是本地 MAC 地址。对于全局地址，其前 3 个字节表示 OUI（Organizationally Unique Identifier），是 IEEE 的注册管理机构分配给不同厂家的代码，用以区分不同的厂家。后 3 个字节由厂家自行分配。

全局地址是全球唯一地址，保证设备的 MAC 地址不会产生冲突，WAN 接口的 MAC 地址应该为全局地址。由于本地 MAC 地址不能保证 MAC 地址的唯一性，所以只用于 LAN 接口。局域网环境下，都是 LAN 设备的 WAN 接口以及接入网关设备的 LAN 接口，可保证局域网 MAC 地址不会发生冲突。

模块的 Wi-Fi 网络接口，烧录了移远通信 OUI 的全局 MAC 地址。其他的网络接口，包括 ETH 网络接口和 USB 网络接口，均使用 Linux 随机生成的本地 MAC 地址。作为 LAN 接口时，不会出现 MAC 地址冲突，但作为 WAN 接口时无法避免 MAC 地址冲突（随机 MAC 地址碰撞的概率极小）。在无法确定是否会产生 MAC 地址冲突的场景下，推荐用户在 ETH 和 USB 网络接口作为 WAN 接口时，使用用户自行向 IEEE 购买的 MAC 地址，并配置 QL_INTF 的 MAC 参数。

3.11.2. 禁用 QL_NF

由于 QL_NF 框架无法满足某些特殊需求，当用户有特殊需求时可禁用 QL_NF。

SDK 中提供了较为底层的接口（如拨号接口，详见文档 [3]），结合 Linux 的网络管理，并通过清空 `/etc/ql_nf_preload.conf` 文件禁用 QL_NF，开发所需要的网络管理程序。

4 场景案例

本章介绍如何在各个典型应用场景下配置 QL_NF 规则，示例中的域名、IP 地址或者 MAC 地址等信息并非真实存在，只为举例说明，用户可以根据实际信息进行设置。

推荐的 QL_NF 配置规则步骤如下：

步骤一 若使用数据拨号，请设置默认承载的 APN 信息（详见第 3.8.3 章），用户也可以在应用程序中使用数据拨号 API 设置（详见文档 [3]）。

步骤二 配置 LAN 相关的 NF_INTF（详见第 3.6 章）。

步骤三 启动 DHCP 和 DNS 服务（详见第 3.6.1 章和第 3.6.2 章）。

步骤四 配置 WAN 相关的 NF_INTF（详见第 3.2 章）。如果 WAN 接口是数据拨号的接口，用户也可以只创建数据拨号的 NF_INTF，由应用程序调用数据拨号的 API 去管理数据拨号。

步骤五 配置 ACL 规则（详见第 3.9 章）。

用户可结合第 2 章的基本概念，了解网络框架，参考上述的步骤，完成 QL_NF 规则配置。完成调试后，再将规则固化到 `/etc/ql_nf_preload.conf` 文件。各个场景的数据转发测试，可以在模块上使用 `tcpdump` 工具，通过抓取各个网络接口收到的数据包，验证分析转发规则是否生效。

备注

在开发过程中，可能需要前期的验证工作，在场景三（第 4.3 章）中，举例说明了如何使用装有 Ubuntu 系统的 PC 进行模拟测试。

4.1. 场景一

4.1.1. 场景需求

● 配置环境

1. (U)SIM 卡：(U)SIM 卡签约一路 APN，名称为 `apn_name`，该路 APN 作为默认承载。
2. WAN 接口：使用签约的 APN 进行数据拨号。
3. LAN 接口：Wi-Fi AP 模式对应的接口、ETH 对应的接口和 USB ECM 对应的接口。

4. 局域网：使用 Wi-Fi、ECM 和 ETH 接入的设备，位于一个局域网，网段为 192.168.225.0/24，网关地址为 192.168.225.1。

● 转发策略

- 1. 模块本地程序使用数据拨号访问网络。
- 2. LAN 设备使用数据拨号访问网络。

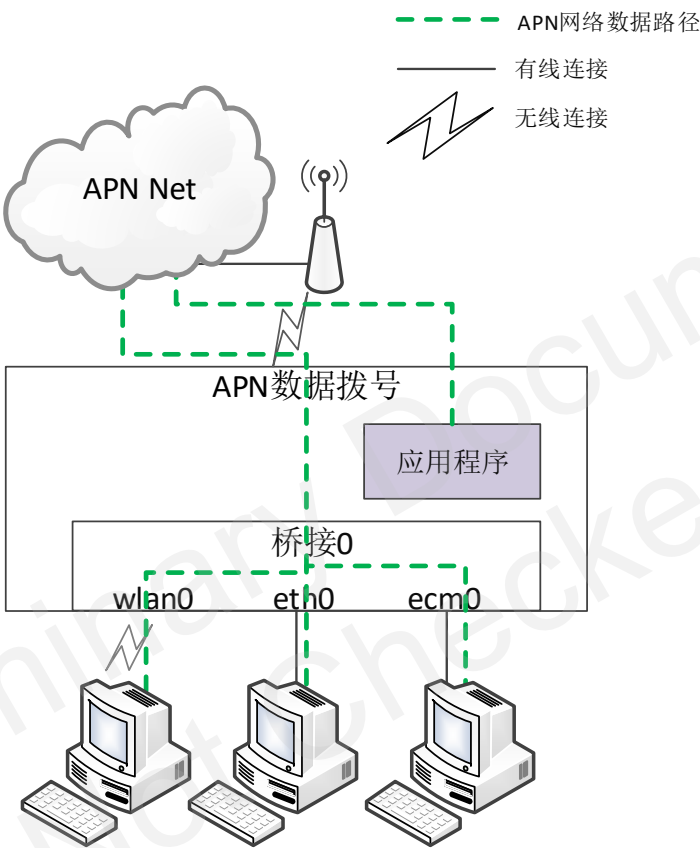


图 7：场景一 Linux 网络拓扑图

4.1.2. QL_NF 组网

根据该场景的需求，NF_INTF 定义如下：

NF_INTF	类型	Linux 网络接口名称	描述	地址	WAN/LAN	是否开启 DHCP 服务
intf_lan	bridge	bridge0	管理局域网 192.168.255.0/24	192.168.225.1/24	LAN	是
intf_eth	com_dev	eth0	以太网接口	/	LAN	/

<i>intf_wifi_ap0</i>	<i>wifi_ap</i>	<i>wlan0</i>	Wi-Fi AP 模式对应的 NF_INTF	/	LAN	/
<i>intf_usb</i>	<i>usb_dev</i>	<i>ecm0</i>	USB ECM 接口对应的 NF_INTF	/	LAN	/
<i>intf_data</i>	<i>data_call</i>	动态信息, 根据拨号结果返回	基于 APN 的数据拨号的 NF_INTF	数据拨号成功后的信息	WAN	/

根据场景需求，转发规则如下：

ACL 名称	数据流来源 NF_INTF	数据流匹配规则	数据流目的 NF_INTF	描述
<i>acl_app_apn</i>	<i>intf_local</i>	所有数据流	<i>intf_data</i>	运行在模块上的应用程序,使用基于 APN 的数据拨号访问网络
<i>acl_lan_apn</i>	<i>intf_lan</i>	所有数据流	<i>intf_data</i>	接入局域网的 LAN 设备, 通过基于 APN 的数据拨号访问网络

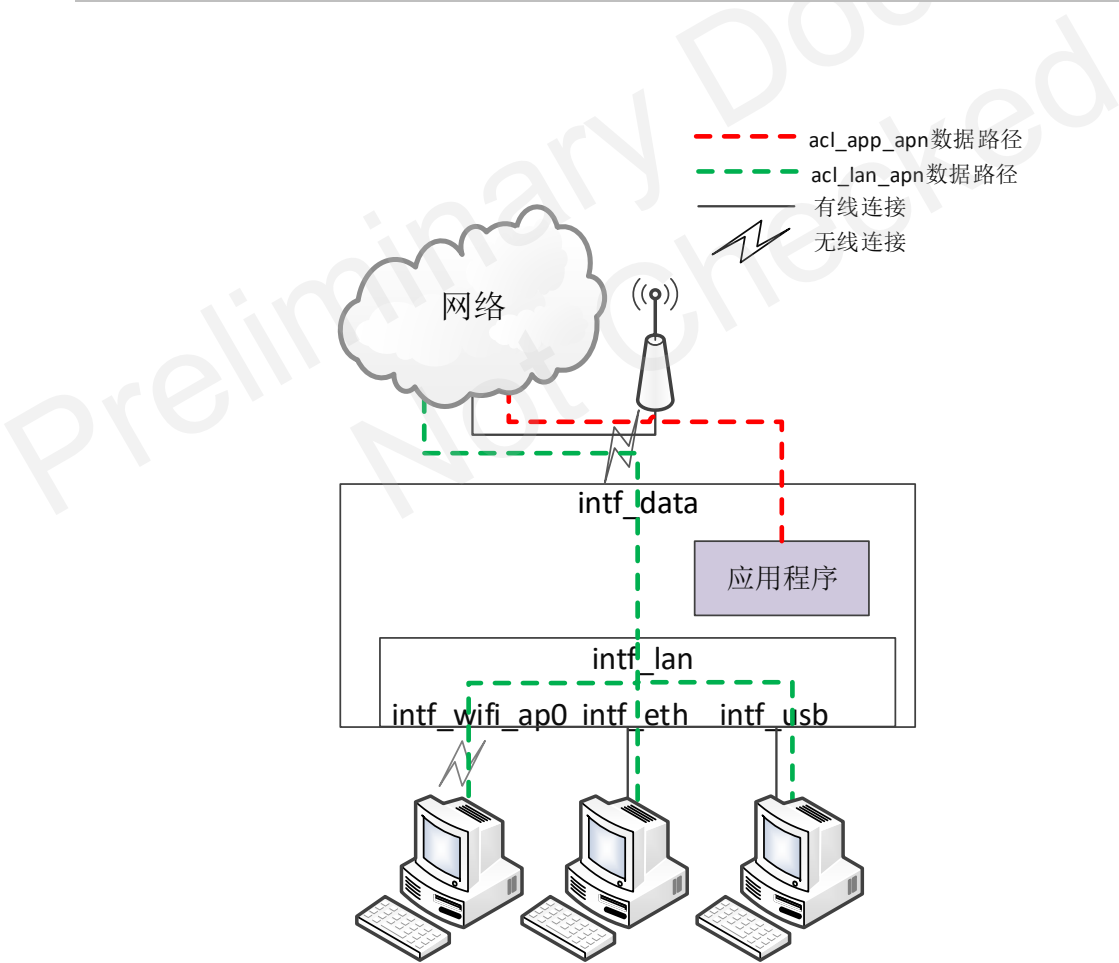


图 8：场景一 QL_NF 网络拓扑图

4.1.3. 规则配置

步骤一 设置默认承载 APN 信息为 SIM 卡签约 APN 信息。

```
set networks attach_apn apn_name=apn_name ip_ver=4
```

步骤二 创建 LAN NF_INTF，创建桥接类型的 NF_INTF 用于管理局域网，并把 LAN NF_INTF 加入桥接 NF_INTF。

```
add interface intf_usb type=usb_dev
add interface intf_eth type=com_dev ifname=eth0
add interface intf_wifi_ap0 type=wifi_ap index=0
add interface intf_lan type=bridge ifname=br0
set interface intf_lan proto=static conn=auto ip=192.168.225.1 netmask=255.255.255.0
set interface intf_lan addif=intf_usb
set interface intf_lan addif=intf_eth
set interface intf_lan addif=intf_wifi_ap0
```

步骤三 启动 DNS 代理服务和局域网的 DHCP 服务，DHCP 服务需要绑定对应的桥接 NF_INTF。

```
add dhcp_srv dhcp_lan intf=intf_lan
set dhcp_srv dhcp_lan ip_start=192.168.225.100 ip_end=192.168.225.200
netmask=255.255.255.0 gateway=192.168.225.1 dnsp=192.168.225.1 lease_time=86400
start dhcp_srv dhcp_lan
add dns_srv dns_lan
start dns_srv dns_lan
```

步骤四 创建 WAN NF_INTF。

```
add data_call 1
set data_call 1 apn_name= apn_name ip_ver=4 reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 1
add interface intf_data type=data_call call_id=1
```

步骤五 配置转发规则。

```
add acl acl_app_apn sintf=intf_local dintf=intf_data
add acl acl_lan_apn sintf=intf_lan dintf=intf_data
```

步骤六 Wi-Fi 相关的配置。详见文档 [2]。

4.1.4. 场景测试

将上述章节的所有命令拷贝到`/etc/ql_nf_preload.conf`文件，重启设备后等待模块网络注册成功，运行`ql_cmd nf get interface all`命令，桥接 NF_INTF 应为 *connected* 状态。等待 WAN NF_INTF 状态变为 *connected* 状态。验证以下内容：

1. 模块上的应用程序，可以访问网络。
2. LAN 设备可以获取 IP 地址并访问网络。

4.2. 场景二

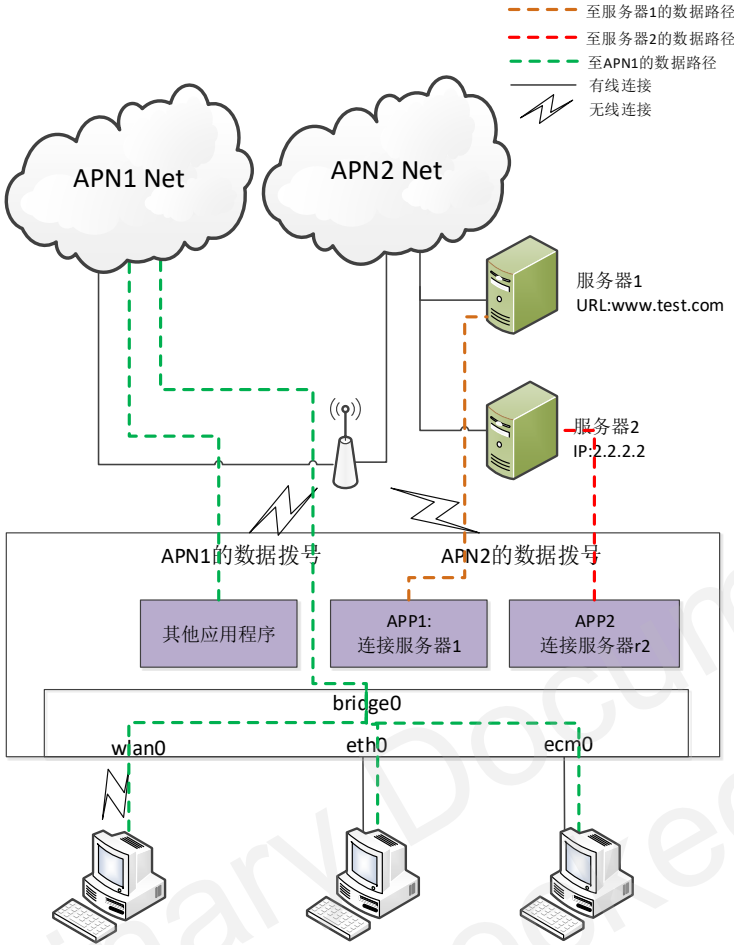
4.2.1. 场景需求

● 配置环境

1. SIM 卡：签约两路 APN 信息，APN1 名称为 `apn_name1`，APN2 名称为 `apn_name2`，其中 APN1 为作为默认承载 APN。
2. WAN 接口：基于 APN1 和 APN2 的数据拨号接口。
3. LAN 接口：Wi-Fi AP 模式对应的接口、ETH 对应的接口和 USB ECM 对应的接口。
4. 局域网：使用 Wi-Fi、USB ECM 和 ETH 接入的设备位于一个局域网内，网段为 192.168.225.0/24，网关地址为 192.168.225.1。

● 转发策略

1. 模块本地程序 APP1 通过 APN2 的数据拨号以域名 `www.test.com` 方式访问网络服务。
2. 模块本地程序 APP2 通过 APN2 的数据拨号以 IP 地址 `2.2.2.2` 方式访问网络服务。
3. 模块其他本地程序通过 APN1 的数据拨号访问网络。
4. LAN 设备通过 APN1 的数据拨号访问网络。



4.2.2. QL_NF 组网

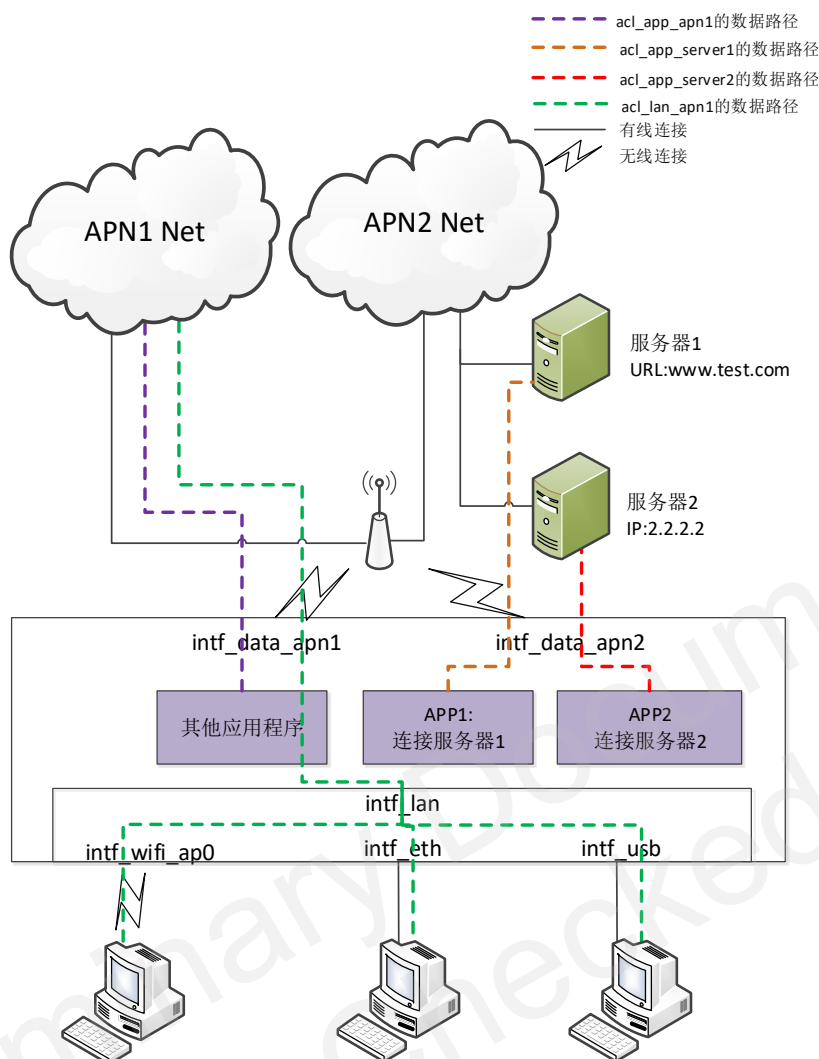
根据该场景的需求，NF_INTF 定义如下：

NF_INTF	类型	Linux 网络接口名称	描述	地址	WAN/LAN	是否开启 DHCP 服务
intf_lan	bridge	bridge0	管理局域网 192.168.255.0/24	192.168.225.1/24	LAN	是
intf_eth	com_dev	eth0	以太网接口的 NF_INTF	/	LAN	/
intf_wifi_ap0	wifi_ap	wlan0	Wi-FiAP 模式对应的 NF_INTF	/	LAN	/
intf_usb	usb_dev	ecm0	USB ECM 对应的 NF_INTF	/	LAN	/
intf_data_	data_call	数据拨号成	APN1 的数据拨	数据拨号成功后	WAN	/

apn1		功后的信息	号的 NF_INTF	的信息	
intf_data_apn2	data_call	数据拨号成功后的信息	APN2 的数据拨号的 NF_INTF	数据拨号成功后的信息	WAN /

根据场景需求，转发规则如下：

ACL 名称	数据流来源 NF_INTF	数据流匹配 规则	数据流目的 NF_INTF	描述
acl_app_server1	intf_local	域 名 为 www.test.com	intf_data_apn2	运行在模块上的应用程序，通过 APN2 的数据拨号以域名访问方式访问网络服务。
acl_app_server2	intf_local	目的 IP 地址 为 2.2.2.2	intf_data_apn2	运行在模块上的应用程序，通过 APN2 的数据拨号以域名 www.test.com 方式访问网络服务。
acl_app_apn1	intf_local	所有数据流	intf_data_apn1	运行在模块的应用程序，通过 APN2 的数据拨号以 IP 地址 2.2.2.2 方式访问网络服务。
acl_lan_apn1	intf_lan	所有数据流	intf_data_apn1	LAN 设备通过 APN1 的数据拨号访问网络。



4.2.3. 规则配置

步骤一 设置默认承载 APN 信息为 SIM 卡签约 APN1 信息。

```
set networks attach_apn apn_name=testapn1 ip_ver=4
```

步骤二 创建 LAN NF_INTF，创建桥接类型的 NF_INTF 用于管理局域网，并把 LAN NF_INTF 加入桥接 NF_INTF。

```
add interface intf_usb type=usb_dev
add interface intf_eth type=com_dev ifname=eth0
add interface intf_wifi_ap0 type=wifi_ap index=0
add interface intf_lan type=bridge ifname=br0
```

```
set interface intf_lan proto=static conn=auto ip=192.168.225.1 netmask=255.255.255.0
set interface intf_lan addif=intf_usb
set interface intf_lan addif=intf_eth
set interface intf_lan addif=intf_wifi_ap0
```

步骤三 启动 DNS 代理服务和局域网的 DHCP 服务，DHCP 服务需要绑定对应的桥接 NF_INTF。

```
add dhcp_srv dhcp_lan intf=intf_lan
set      dhcp_srv      dhcp_lan      ip_start=192.168.225.100      ip_end=192.168.225.200
netmask=255.255.255.0 gateway=192.168.225.1 dnsp=192.168.225.1 lease_time=86400
start dhcp_srv dhcp_lan
add dns_srv dns_lan
start dns_srv dns_lan
```

步骤四 创建 WAN NF_INTF。

```
add data_call 1
set      data_call      1      apn_name=testapn1      ip_ver=4      reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 1
add data_call 2
set      data_call      2      apn_name=testapn2      ip_ver=4      reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 2
add interface intf_data_apn1 type=data_call call_id=1
add interface intf_data_apn2 type=data_call call_id=2
```

步骤五 配置转发规则。

```
add acl acl_app_server1 sintf=intf_local url=test.com dintf=intf_data_apn2
add acl acl_app_server2 sintf=intf_local dip=2.2.2.2 dintf=intf_data_apn2
add acl acl_app_apn1 sintf=intf_local dintf=intf_data_apn1
add acl acl_lan_apn1 sintf=intf_lan dintf=intf_data_apn1
```

4.2.4. 场景测试

将上述章节的所有命令拷贝到 `/etc/ql_nf_preload.conf` 文件，重启设备后等待模块网络注册成功。运行 `ql_cmd nf get interface all` 命令，桥接 NF 接口应为 `connected` 状态。等待 WAN NF 接口状态变为 `connected` 状态。可以做以下验证：

1. 在模块上，访问 `test.com`，通过 `intf_data_apn2` 收发数据。
2. 在模块上，访问 `2.2.2.2`，通过 `intf_data_apn2` 收发数据。
3. 在模块上，通过 `intf_data_apn1` 访问网络。
4. LAN 设备，通过 `intf_data_apn1` 访问网络。

4.3. 场景三

4.3.1. 场景需求

- 配置环境

1. (U)SIM 卡：签约两路 APN 信息，APN1 名称为 testapn1；APN2 名称为 testapn2，其中 APN1 作为默认承载 APN。
2. WAN 接口：基于 APN1 和 APN2 的数据拨号接口。
3. LAN 接口：Wi-Fi AP 对应的接口，基于 ETH 且 VLAN ID 分别为 100 和 200 的接口。
4. 局域网：使用 ETH VID 100 和 Wi-Fi 接入的设备位于一个局域网，网段为 192.168.100.0/24；使用 ETH VID 200 接入的设备位于一个局域网，网段为 192.168.200.0/24。

- 转发策略

1. 模块本地程序使用 APN2 的数据拨号访问域名为 www.test.com 的服务器。
2. 模块本地程序使用 APN1 的数据拨号访问其他网络服务。
3. 通过 ETH VID 100 接入的设备使用基于 APN1 的数据拨号访问网络。
4. 通过 ETH VID 200 接入的设备使用基于 APN2 的数据拨号访问网络。
5. 通过 Wi-Fi 接入的设备，通过基于 APN1 的数据拨号访问网络。

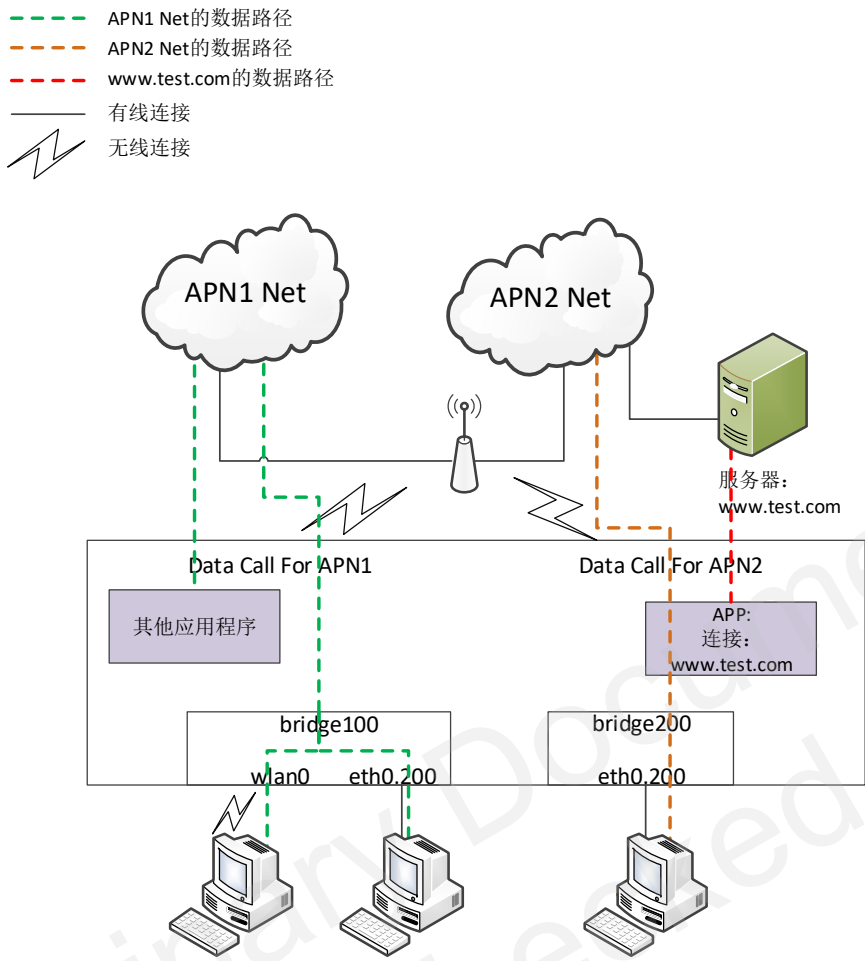


图 11: 场景三 Linux 网络拓扑图

4.3.2. QL_NF 组网

根据该场景的需求，NF_INTF 定义如下：

NF_INTF	类型	Linux 网络接口名称	描述	地址	WAN/ LAN	是否开启 DHCP 服务
intf_lan100	bridge	bridge100	管理局域网 192.168.100.0/24	192.168.100.1/24	LAN	是
intf_lan100	bridge	bridge200	管理局域网 192.168.200.0/24	192.168.200.1/24	LAN	是
intf_eth	com_dev	eth0	以太网接口的 NF_INTF	/	LAN	/

intf_100	vdev	eth0.100	基于以太网接口的 VLAN ID 为 / 100 的 NF_INTF		LAN	/
intf_200	vdev	eth0.200	基于以太网接口的 VLAN ID 为 / 200 的 NF_INTF		LAN	/
intf_wifi_ap0	wifi_ap	wlan0	Wi-Fi AP 模式对应的 NF_INTF /		LAN	/
intf_data_apn1	data_call	数据拨号成功后的信息	APN1 的数据拨号的 NF_INTF	数据拨号成功后的信息	WAN	/
intf_data_apn2	data_call	数据拨号成功后的信息	APN2 的数据拨号的 NF_INTF	数据拨号成功后的信息	WAN	/

根据场景需求，转发规则如下：

ACL 名称	数据流来源 NF_INTF	数据流匹配规则	数据流目的 NF_INTF	描述
acl_lan100_apn1	intf_lan100	所有数据流	intf_data_apn1	192.168.100.0/24 网段的局域网，通过 APN1 的数据拨号访问网络
acl_lan200_apn2	intf_lan200	所有数据流	intf_data_apn2	192.168.200.0/24 网段的局域网，通过 APN1 的数据拨号访问网络
acl_app_server	intf_local	url=test.com	intf_data_apn2	运行在模块的应用程序，通过 APN2 的数据拨号访问域名为 www.test.com 的网络服务。
acl_app_apn1	intf_lan	所有数据流	intf_data_apn1	运行在模块的应用程序，通过 APN1 的数据拨号访问网络。

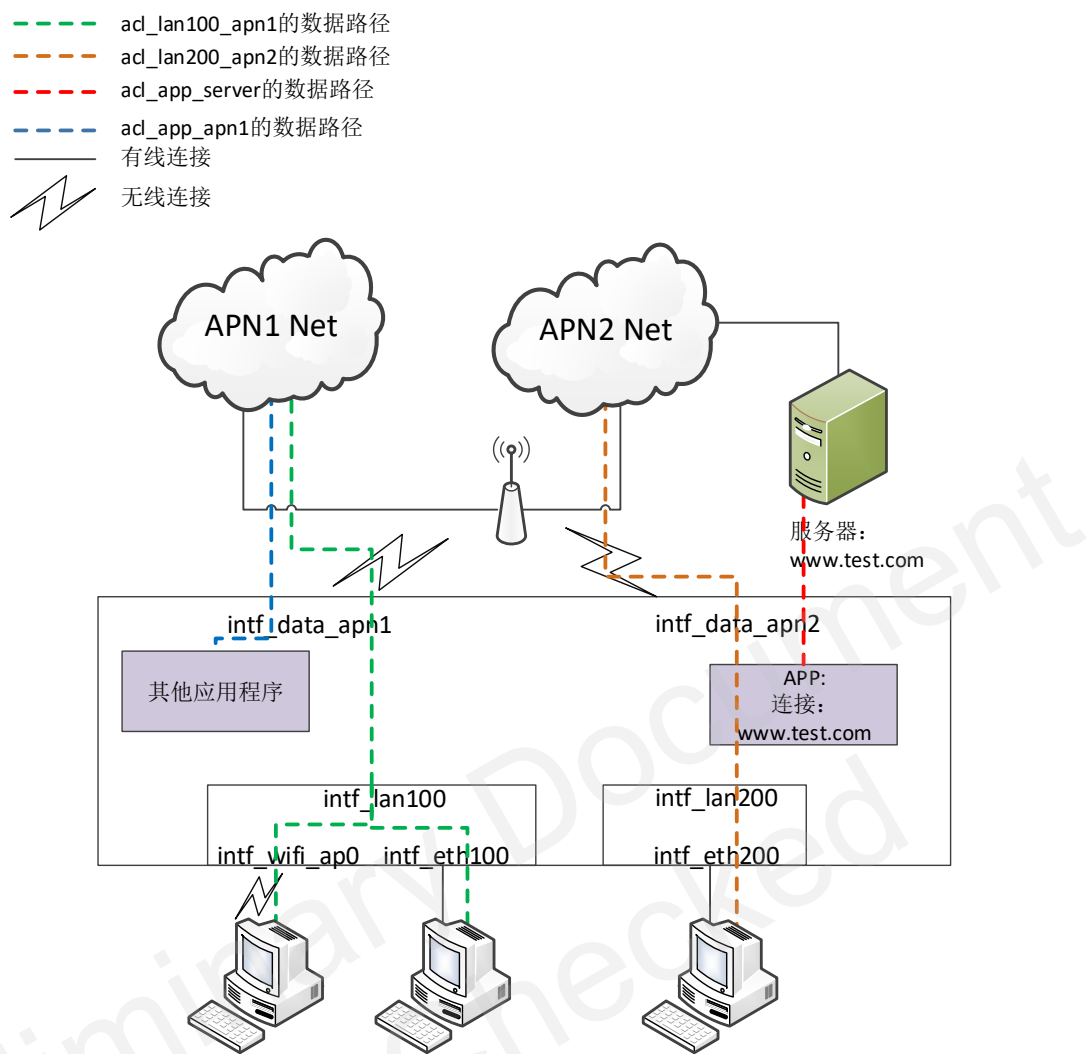


图 12: 场景三 QL_NF 网络拓扑图

4.3.3. 规则配置

步骤一 设置默认承载 APN 信息为 SIM 卡签约 APN1 信息。

```
set networks attach_apn apn_name=testapn1 ip_ver=4
```

步骤二 创建 LAN NF_INTF，创建桥接类型的 NF_INTF 用于管理局域网，并把 LAN NF_INTF 加入桥接 NF_INTF。

```
add interface intf_eth type=com_dev ifname=eth0
add interface intf_eth100 type=vdev master=intf_eth vlan=100
add interface intf_eth200 type=vdev master=intf_eth vlan=200
add interface intf_wifi_ap0 type=wifi_ap index=0
```

```
add interface intf_lan100 type=bridge ifname=bridge100
set interface intf_lan100 proto=static conn=auto ip=192.168.100.1 netmask=255.255.255.0
set interface intf_lan100 addif=intf_eth100
set interface intf_lan100 addif=intf_wifi_ap0

add interface intf_lan200 type=bridge ifname=bridge200
set interface intf_lan200 proto=static conn=auto ip=192.168.200.1 netmask=255.255.255.0
set interface intf_lan200 addif=intf_eth200
```

步骤三 启动 DNS 代理服务和局域网的 DHCP 服务，DHCP 服务需要绑定对应的桥接 NF_INTF。

```
add dhcp_srv dhcp_lan100 intf=intf_lan100
set dhcp_srv dhcp_lan100 ip_start=192.168.100.100 ip_end=192.168.100.200
netmask=255.255.255.0 gateway=192.168.100.1 dnsp=192.168.100.1 lease_time=86400
start dhcp_srv dhcp_lan100
add dhcp_srv dhcp_lan200 intf=intf_lan200
set dhcp_srv dhcp_lan200 ip_start=192.168.200.100 ip_end=192.168.200.200
netmask=255.255.255.0 gateway=192.168.200.1 dnsp=192.168.200.1 lease_time=86400
start dhcp_srv dhcp_lan200
add dns_srv dns_lan
start dns_srv dns_lan
```

步骤四 创建 WAN NF_INTF。

```
add data_call 1
set data_call 1 apn_name=testapn1 ip_ver=4 reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 1
add data_call 2
set data_call 2 apn_name=testapn2 ip_ver=4 reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 2
add interface intf_data_apn1 type=data_call call_id=1
add interface intf_data_apn2 type=data_call call_id=2
```

步骤五 配置转发规则。

```
add acl acl_lan100_apn1 sintf=intf_lan100 dintf=intf_data_apn1
add acl acl_lan200_apn2 sintf=intf_lan200 dintf=intf_data_apn2
add acl acl_app_server sintf=intf_local url=test.com dintf=intf_data_apn2
add acl acl_app_apn1 sintf=intf_local dintf=intf_data_apn1
```

4.3.4. 场景测试

本章节使用真实环境测试验证场景三的 QL_NF 规则。下表是测试使用的设备。

表 18：设备信息介绍

设备	介绍
LTE OPEN-EVB	开发板
(U)SIM 卡	普通中国联通(U)SIM 卡，支持多路拨号
模块	支持中国联通频段的模块，安装到 EVB
YT8521 模块	PHY 模块，安装到 EVB
FC30R 模组	Wi-Fi 模块，安装到 EVB
千兆以太网交换机	通过以太网连接 PC1、PC2 和 PHY 模块。
PC1	运行 ubuntu1804 版本的笔记本电脑，接入 VID 100 的 VLAN
PC2	运行 ubuntu1804 版本的笔记本电脑，接入 VID 200 的 VLAN
手机	支持 Wi-Fi 功能的安卓手机

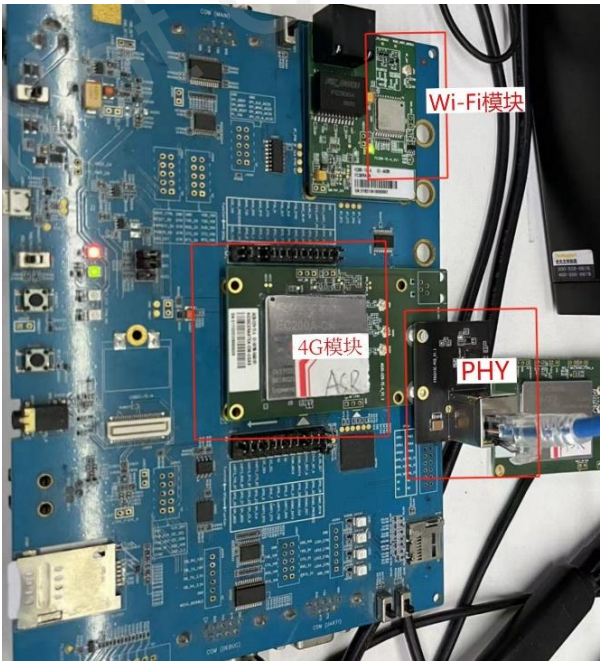


图 13：EVB 环境

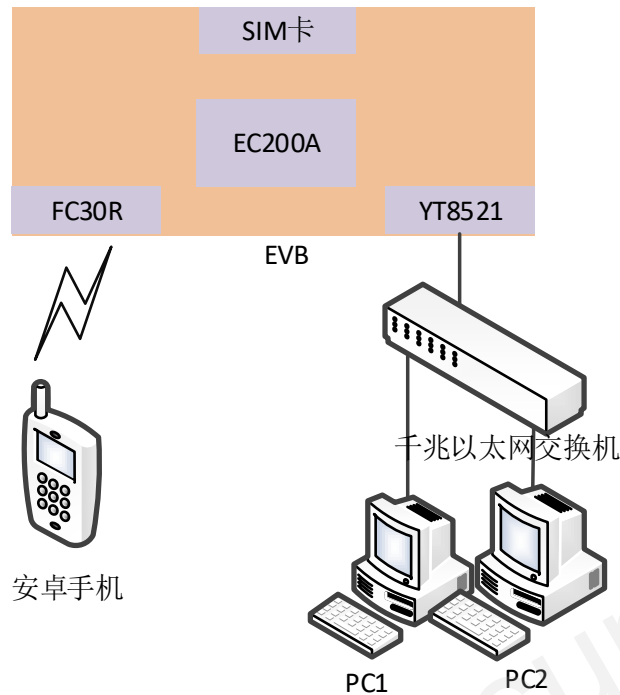


图 14: 测试组网

4.3.4.1. 模块配置

根据测试环境的情况，修改 APN1 和 APN2 的名称为 3GNET 和 testapn（测试使用的是普通(U)SIM 卡，支持使用任意 APN 名称进行数据拨号，实际使用时，请替换为(U)SIM 卡签约的 APN 名称）。修改域名 www.test.com 为可访问的域名 www.quectel.com。修改后的 NF 规则如下：

```
set networks attach_apn apn_name=3GNET ip_ver=4
add interface intf_eth type=com_dev ifname=eth0
add interface intf_eth100 type=vdev master=intf_eth vlan=100
add interface intf_eth200 type=vdev master=intf_eth vlan=200
add interface intf_wifi_ap0 type=wifi_ap index=0
add interface intf_lan100 type=bridge ifname=bridge100
set interface intf_lan100 proto=static conn=auto ip=192.168.100.1 netmask=255.255.255.0
set interface intf_lan100 addif=intf_eth100
set interface intf_lan100 addif=intf_wifi_ap0
add interface intf_lan200 type=bridge ifname=bridge200
set interface intf_lan200 proto=static conn=auto ip=192.168.200.1 netmask=255.255.255.0
set interface intf_lan200 addif=intf_eth200
add dhcp_srv dhcp_lan100 intf=intf_lan100
set dhcp_srv dhcp_lan100 ip_start=192.168.100.100 ip_end=192.168.100.200
netmask=255.255.255.0 gateway=192.168.100.1 dnsp=192.168.100.1 lease_time=86400
start dhcp_srv dhcp_lan100
```

```
add dhcp_srv dhcp_lan200 intf=intf_lan200
set    dhcp_srv    dhcp_lan200    ip_start=192.168.200.100    ip_end=192.168.200.200
netmask=255.255.255.0 gateway=192.168.200.1 dnsp=192.168.200.1 lease_time=86400
start dhcp_srv dhcp_lan200
add dns_srv dns_lan
start dns_srv dns_lan
add data_call 1
set    data_call    1    apn_name=3GNET    ip_ver=4    reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 1
add data_call 2
set    data_call    2    apn_name=testapn    ip_ver=4    reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 2
add interface intf_data_apn1 type=data_call call_id=1
add interface intf_data_apn2 type=data_call call_id=2
add acl acl_lan100_apn1 sintf=intf_lan100 dintf=intf_data_apn1
add acl acl_lan200_apn2 sintf=intf_lan200 dintf=intf_data_apn2
add acl acl_app_server sintf=intf_local url=quectel.com dintf=intf_data_apn2
add acl acl_app_apn1 sintf=intf_local dintf=intf_data_apn1
```

将 NF 规则写入到文件 `/etc/ql_nf_preload.conf` 中（可以在 SDK 中修改，并重新制作 rootfs），并重启设备。模块启动完成，等待注网成功后，检查数据拨号的 NF_INTF 是否为 `connected` 状态。

```
~ # ql_cmd nf get interface all
name=intf_local status=connected id=1 type=local ifname=lo ip=127.0.0.1 netmask=255.255.255.0 subnet=24
name=intf_eth status=unconfig id=2 type=com_dev ifname=eth0
name=intf_eth100 status=binder id=3 type=vdev ifname=eth0.100
name=intf_eth200 status=binder id=4 type=vdev ifname=eth0.200
name=intf_wifi_ap0 status=none id=5 type=wifi_ap
name=intf_lan100 status=connected id=6 type=bridge ifname=bridge100 ip=192.168.100.1 netmask=255.255.255.0 subnet=24
name=intf_lan200 status=connected id=7 type=bridge ifname=bridge200 ip=192.168.200.1 netmask=255.255.255.0 subnet=24
name=intf_data_apn1 status=connected id=8 type=data_call ifname=rmnet_data0 ip=10.239.80.105 netmask=255.255.255.252 subnet=30 gat
name=intf_data_apn2 status=connected id=9 type=data_call ifname=rmnet_data1 ip=10.238.27.155 netmask=255.255.255.248 subnet=29 gat
```

4.3.4.2. Wi-Fi 配置(wifi 接口部分目前接口暂时未开发)

在模块上运行 API 测试程序 `ql_sdk_api_test`，配置 Wi-Fi。设置步骤如下：

1. 运行 `ql_sdk_api_test`，选择 Wi-Fi，进入 Wi-Fi API 选项。
2. 选择 `ql_wifi_init` 进行初始化。
3. 选择 `ql_wifi_enable` 使用 Wi-Fi。
4. 选择 `ql_wifi_work_mode_set`，设置模式为“1-AP0”模式。
5. 选择 `ql_wifi_ap_ssid_set`，设置 AP0 的 SSID 为“quectel_test”。
6. 选择 `ql_wifi_ap_auth_set`，设置 AP0 的加密参数：WPA_WPA2_PSK_BOTH, 0-AUTO, 11111111。
7. 选择 `ql_wifi_ap_start`，打开 AP0。

手机选择 SSID `quectel_test`，并连接 Wi-Fi，连接后能正常访问网络。

4.3.4.3. PC1 配置

断开 PC1 上其他的网络连接，并输入以下指令停止系统的网络管理功能。

```
sudo /etc/init.d/networking stop
```

然后输入以下命令，配置基于以太网的、VLAN ID 为 100 的网络连接。

```
sudo vconfig add eth0 100
sudo ifconfig eth0.100 up
sudo dhclient eth0.100
```

上述命令运行成功后，eth0.100 网卡可以获取到 IP 地址，PC1 可以访问网络。

4.3.4.4. PC2 配置

断开 PC2 上其他的网络连接，并输入以下指令停止系统的网络管理功能。

```
sudo /etc/init.d/networking stop
```

然后输入以下命令，配置基于以太网的、VID 为 200 的网络连接。

```
sudo vconfig add eth0 200
sudo ifconfig eth0.200 up
sudo dhclient eth0.200
```

上述命令运行成功后，eth0.100 网卡可以获取到 IP 地址，PC2 可以访问网络。

4.3.4.5. 数据转发测试

通过 **ql_cmd nf get interface all** 命令可以查看到，intf_data_apn1 对应的 Linux 网络接口是 *ccinet0*；intf_data_apn2 对应的 Linux 网络接口是 *ccinet1*。

在模块中，通过 **tcpdump** 在 *ccinet0* 和 *ccinet1* 上抓取数据包，然后在模块、PC1、PC2 和手机上分别访问网络，查看网络访问路径是否按照 ACL 指定的路径传输。验证方法如下：

在模块运行以下命令抓取 *ccinet0* 和 *ccinet1* 上传输的报文，并保存为文件。测试结束后，把文件上传到 PC，并使用 **wireshark** 工具分析网络报文是否是在相应的 Linux 网络接口上传输。

```
tcpdump -i ccinet0 -w /tmp/ccinet 0 &
tcpdump -i ccinet1 -w /tmp/ccinet1 &
```

可验证以下场景：

1. 通过 *ccinet0* 抓取包含 PC1 访问网络的报文；通过 *ccinet1* 抓取不包含 PC1 访问网络的报文。
2. 通过 *ccinet1* 抓取包含 PC2 访问网络的报文；通过 *ccinet0* 抓取不包含 PC2 访问网络的报文。
3. 通过 *ccinet1* 抓取包含模块访问 www.quectel.com 的报文（如使用 ping 命令访问）；通过 *ccinet1* 抓取不包含模块访问 www.quectel.com 的报文。
4. 通过 *ccinet0* 抓取包含模块访问其他网络的报文（如使用 ping 命令访问 www.github.com）；通过 *ccinet1* 抓取不包含模块访问其他网络的报文。
5. 通过 *ccinet0* 抓取到包含手机访问网络的报文；通过 *ccinet1* 抓取不包含手机访问网络的报文。

上述 3、4 场景的验证方法如下：

在模块上使用 tcpdump 开启抓包后，使用 ping 命令分别访问 www.quectel.com 和 www.github.com。然后终止 tcpdump 进程，上传对应的报文文件 */tmp/ccinet0.pcap* 和 */tmp/ccinet1.pcap* 到 PC。

在 PC 上，用 wireshark 打开 *ccinet0.pcap*，可以看到访问 www.github.com 的报文。

19	2021-07-16 14:11:53.618703	10.236.223.203	218.104.78.2	76	DNS	Standard query 0x6eb7 A www.github.com
20	2021-07-16 14:11:53.623349	10.236.223.203	218.104.78.2	76	DNS	Standard query 0x193c AAAA www.github.com
21	2021-07-16 14:11:53.692437	218.104.78.2	10.236.223.203	106	DNS	Standard query response 0x6eb7 A www.github.com CNAME github.com A 13.229.188.59
22	2021-07-16 14:11:53.704114	218.104.78.2	10.236.223.203	90	DNS	Standard query response 0x193c AAAA www.github.com CNAME github.com
23	2021-07-16 14:11:53.712609	10.236.223.203	13.229.188.59	100	ICMP	Echo (ping) request id=0xca0a, seq=0/0, ttl=64 (reply in 24)
24	2021-07-16 14:11:54.087139	13.229.188.59	10.236.223.203	100	ICMP	Echo (ping) reply id=0xca0a, seq=0/0, ttl=44 (request in 23)
25	2021-07-16 14:11:54.713032	10.236.223.203	13.229.188.59	100	ICMP	Echo (ping) request id=0xca0a, seq=1/256, ttl=64 (no response found!)
26	2021-07-16 14:11:55.713551	10.236.223.203	13.229.188.59	100	ICMP	Echo (ping) request id=0xca0a, seq=2/512, ttl=64 (reply in 27)
27	2021-07-16 14:11:56.087119	13.229.188.59	10.236.223.203	100	ICMP	Echo (ping) reply id=0xca0a, seq=2/512, ttl=44 (request in 26)
28	2021-07-16 14:11:56.714185	10.236.223.203	13.229.188.59	100	ICMP	Echo (ping) request id=0xca0a, seq=3/768, ttl=64 (reply in 29)
29	2021-07-16 14:11:57.120812	13.229.188.59	10.236.223.203	100	ICMP	Echo (ping) reply id=0xca0a, seq=3/768, ttl=44 (request in 28)
30	2021-07-16 14:11:57.714856	10.236.223.203	13.229.188.59	100	ICMP	Echo (ping) request id=0xca0a, seq=4/1024, ttl=64 (reply in 31)
31	2021-07-16 14:11:58.084261	13.229.188.59	10.236.223.203	100	ICMP	Echo (ping) reply id=0xca0a, seq=4/1024, ttl=44 (request in 30)

在 PC 上，使用 wireshark 打开 *ccinet1.pcap*，可以看到访问 www.quectel.com 的报文。

1	2021-07-16 14:11:42.998877	10.234.43.189	218.104.78.2	77	DNS	Standard query 0x7212 A www.quectel.com
2	2021-07-16 14:11:43.002773	10.234.43.189	218.104.78.2	77	DNS	Standard query 0x8d49 AAAA www.quectel.com
3	2021-07-16 14:11:43.055036	218.104.78.2	10.234.43.189	93	DNS	Standard query response 0x7212 A www.quectel.com A 47.101.71.206
4	2021-07-16 14:11:43.101272	218.104.78.2	10.234.43.189	77	DNS	Standard query response 0x8d49 AAAA www.quectel.com
5	2021-07-16 14:11:43.109658	10.234.43.189	47.101.71.206	100	ICMP	Echo (ping) request id=0xc90a, seq=0/0, ttl=64 (reply in 6)
6	2021-07-16 14:11:43.183802	47.101.71.206	10.234.43.189	100	ICMP	Echo (ping) reply id=0xc90a, seq=0/0, ttl=52 (request in 5)
7	2021-07-16 14:11:44.110036	10.234.43.189	47.101.71.206	100	ICMP	Echo (ping) request id=0xc90a, seq=1/256, ttl=64 (reply in 8)
8	2021-07-16 14:11:44.206570	47.101.71.206	10.234.43.189	100	ICMP	Echo (ping) reply id=0xc90a, seq=1/256, ttl=52 (request in 7)
9	2021-07-16 14:11:45.110418	10.234.43.189	47.101.71.206	100	ICMP	Echo (ping) request id=0xc90a, seq=2/512, ttl=64 (reply in 10)
10	2021-07-16 14:11:45.166508	47.101.71.206	10.234.43.189	100	ICMP	Echo (ping) reply id=0xc90a, seq=2/512, ttl=52 (request in 9)
11	2021-07-16 14:11:46.110835	10.234.43.189	47.101.71.206	100	ICMP	Echo (ping) request id=0xc90a, seq=3/768, ttl=64 (reply in 12)
12	2021-07-16 14:11:46.166319	47.101.71.206	10.234.43.189	100	ICMP	Echo (ping) reply id=0xc90a, seq=3/768, ttl=52 (request in 11)

4.4. 场景四

4.4.1. 场景需求

在一个网络场景中，LAN 侧根据不同的业务类型划分了不同类型的 VLAN，每个 VLAN 都是一个单独的局域网。接入 VLAN 的设备使用静态 IP 地址和静态 ARP 的方式配置网络。同时存在三路数据拨号，不同的数据拨号用于访问不同的业务。

表 19：组网信息

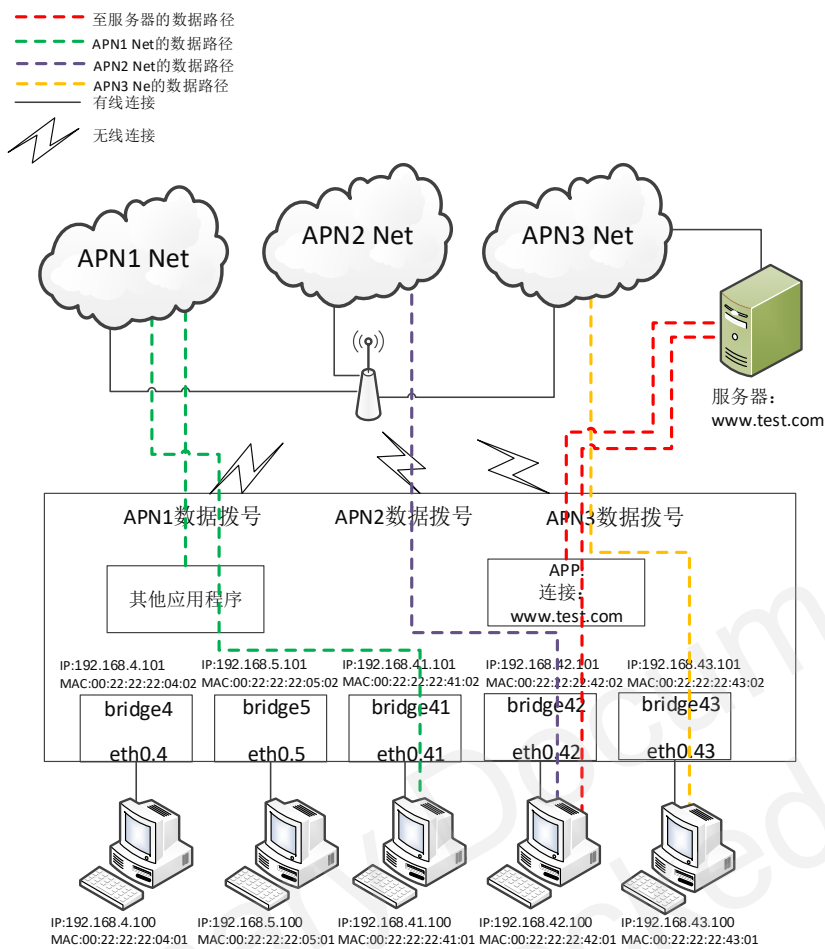
VLAN ID	用途	访问网络	设备信息	模块信息
4	内部通信	无	IP: 192.168.4.100/24 MAC: 00:22:22:22:04:01	IP: 192.168.4.101/24 MAC: 00:22:22:22:04:02
5	内部通信	无	IP: 192.168.5.100/24 MAC: 00:22:22:22:05:01	IP: 192.168.5.101/24 MAC: 00:22:22:22:05:02
41	访问网络	APN1 对应的网络	IP: 192.168.41.100/24 MAC: 00:22:22:22:41:01	IP: 192.168.41.101/24 MAC: 00:22:22:22:41:02
42	访问网络	APN2 对应的网络	IP: 192.168.42.100/24 MAC: 00:22:22:22:42:01	IP: 192.168.42.101/24 MAC: 00:22:22:22:42:02
43	访问网络	APN3 对应的网络	IP: 192.168.43.100/24 MAC: 00:22:22:22:43:01	IP: 192.168.43.101/24 MAC: 00:22:22:22:43:02

● 配置环境

1. (U)SIM 卡：签约三路 APN 信息。APN1 名称为 testapn1；APN2 名称为 testapn2；APN3 名称为 testapn3。其中 APN1 作为默认承载 APN。
2. WAN 接口：三路 APN 对应的数据拨号。
3. LAN 接口：基于 ETH VID 为 4、5、41、42、43 的 VLAN 接口。
4. 局域网：见上方表格。

● 转发策略

1. VLAN ID 为 4 的 VLAN 和 VLAN ID 为 5 的 VLAN 用于局域网内部通信。
2. 使用 VLAN ID 为 41 接入局域网的设备，通过 APN1 访问网络。
3. 使用 VLAN ID 为 42 接入局域网的设备，通过 APN2 访问网络。
4. 使用 VLAN ID 为 43 接入局域网的设备，通过 APN3 访问网络。
5. www.test.com 对应的网络服务位于 APN3 对应的网络。模块上运行的程序和通过 VLAN ID 为 42 接入的 LAN 设备需要通过 APN3 的数据拨号访问该网络服务。
6. 模块上运行的程序使用 APN1 访问其他网络。



4.4.2. QL_NF 组网

根据该场景的需求，NF 接口定义如下：

NF_I NTF	类型	Linux 网 络接口名 称	描述	地址	WAN/ LAN	是否开启 DHCP 服务
intf_la n4	bridge	bridge4	管理 192.168.4.0/24 网段 的局域网	192.168.4.101 /24	LAN	否
intf_la n5	bridge	bridge5	管理 192.168.5.0/24 网段 的局域网	192.168.5.101 /24	LAN	否
intf_la n41	bridge	bridge41	管理 192.168.41.0/24 网 段的局域网	192.168.41.10 1/24	LAN	否
intf_la n42	bridge	bridge42	管理 192.168.42.0/24 网 段的局域网	192.168.42.10 1/24	LAN	否
intf_la n43	bridge	bridge43	管理 192.168.43.0/24 网 段的局域网	192.168.43.10 1/24	LAN	否

<i>intf_eth</i>	<i>com_dev</i>	eth0	以太网接口的 NF_INTF	/	LAN	/
<i>intf_eth4</i>	<i>vdev</i>	eth0.4	基于以太网接口创建的 VID 为 4 的 NF_INTF	/	LAN	/
<i>intf_eth5</i>	<i>vdev</i>	eth0.5	基于以太网接口创建的 VID 为 5 的 NF_INTF	/	LAN	/
<i>intf_eth41</i>	<i>vdev</i>	eth0.41	基于以太网接口创建的 VID 为 41 的 NF_INTF	/	LAN	/
<i>intf_eth42</i>	<i>vdev</i>	eth0.42	基于以太网接口创建的 VID 为 42 的 NF_INTF	/	LAN	/
<i>intf_eth43</i>	<i>vdev</i>	eth0.43	基于以太网接口创建的 VID 为 43 的 NF_INTF	/	LAN	/
<i>intf_data_apn1</i>	<i>data_call</i>	数据拨号成功后的信息	APN1 的数据拨号的接口	数据拨号成功后的信息	WAN	/
<i>intf_data_apn2</i>	<i>data_call</i>	数据拨号成功后的信息	APN2 的数据拨号的接口	数据拨号成功后的信息	WAN	/
<i>intf_data_apn3</i>	<i>data_call</i>	数据拨号成功后的信息	APN3 的数据拨号的接口	数据拨号成功后的信息	WAN	/

根据场景需求，转发规则如下：

ACL 名称	数据流来源 NF_INTF	数据流匹配规则	数据流目的 NF_INTF	描述
<i>acl_app_server</i>	<i>intf_local</i>	url=test.com	<i>intf_data_apn3</i>	运行在模块的应用程序，通过 APN3 的数据拨号访问域名为 www.test.com 的网络服务。
<i>acl_app_apn1</i>	<i>intf_local</i>	所有数据流	<i>intf_data_apn1</i>	运行在模块的应用程序，通过 APN1 的数据拨号访问网络。
<i>acl_lan42_server</i>	<i>intf_lan42</i>	url=test.com	<i>intf_data_apn3</i>	通过 VID 42 的设备，通过 APN3 的数据拨号访问域名为 www.test.com 的网络服务。
<i>acl_lan41_apn1</i>	<i>intf_lan41</i>	所有数据流	<i>intf_data_apn1</i>	通过 VID 41 接入的设备，通过 APN1 的数据拨号访问网络
<i>acl_lan42_apn2</i>	<i>intf_lan42</i>	所有数据流	<i>intf_data_apn2</i>	通过 VID 42 接入的设备，通过 APN2 的数据拨号访问网络
<i>acl_lan43_apn3</i>	<i>intf_lan43</i>	所有数据流	<i>intf_data_apn3</i>	通过 VID 43 接入的设备，通过 APN3 的数据拨号访问网络

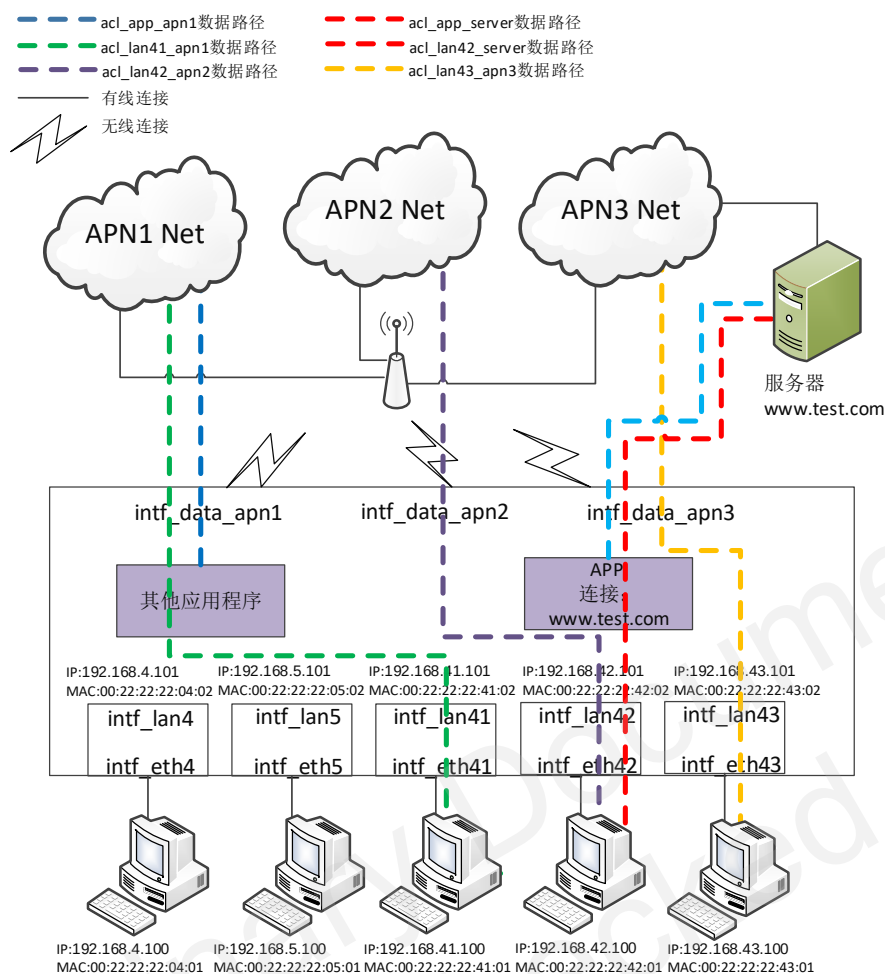


图 16: 场景四 QL_NF 网络拓扑图

4.4.3. 规则配置

步骤一 设置默认承载 APN 信息为(U)SIM 卡签约 APN1 信息。

```
set networks attach_apn apn_name=testapn1 ip_ver=4
```

步骤二 创建 LAN NF_INTF，创建桥接类型的 NF_INTF 用于管理局域网，并把 LAN NF_INTF 加入桥接 NF_INTF。

```
add interface intf_eth type=com_dev ifname=eth0
add interface intf_eth4 type=vdev master=intf_eth vlan=4
add interface intf_eth5 type=vdev master=intf_eth vlan=5
add interface intf_eth41 type=vdev master=intf_eth vlan=41
add interface intf_eth42 type=vdev master=intf_eth vlan=42
add interface intf_eth43 type=vdev master=intf_eth vlan=43
```

```
add interface intf_lan4 type=bridge ifname=bridge4
set interface intf_lan4 proto=static conn=auto ip=192.168.4.101 netmask=255.255.255.0
mac=00:22:22:22:04:02 noarp=1
set interface intf_lan4 addif=intf_eth4

add interface intf_lan5 type=bridge ifname=bridge5
set interface intf_lan5 proto=static conn=auto ip=192.168.5.101 netmask=255.255.255.0
mac=00:22:22:22:05:02 noarp=1
set interface intf_lan5 addif=intf_eth5

add interface intf_lan41 type=bridge ifname=bridge41
set interface intf_lan41 proto=static conn=auto ip=192.168.41.101 netmask=255.255.255.0
mac=00:22:22:22:41:02 noarp=1
set interface intf_lan41 addif=intf_eth41

add interface intf_lan42 type=bridge ifname=bridge42
set interface intf_lan42 proto=static conn=auto ip=192.168.42.101 netmask=255.255.255.0
mac=00:22:22:22:42:02 noarp=1
set interface intf_lan42 addif=intf_eth42

add interface intf_lan43 type=bridge ifname=bridge43
set interface intf_lan43 proto=static conn=auto ip=192.168.43.101 netmask=255.255.255.0
mac=00:22:22:22:43:02 noarp=1
set interface intf_lan43 addif=intf_eth43
```

步骤三 配置静态 ARP 规则。

```
add arp arp4 intf=intf_lan4 ip=192.168.4.102 mac=00:22:22:22:04:02
add arp arp5 intf=intf_lan5 ip=192.168.5.102 mac=00:22:22:22:05:02
add arp arp41 intf=intf_lan41 ip=192.168.41.102 mac=00:22:22:22:41:02
add arp arp42 intf=intf_lan42 ip=192.168.42.102 mac=00:22:22:22:42:02
add arp arp43 intf=intf_lan43 ip=192.168.43.102 mac=00:22:22:22:43:02
```

步骤四 启用 DNS 代理服务。

```
add dns_srv dns_lan
start dns_srv dns_lan
```

步骤五 创建 WAN NF_INTF。

```
add data_call 1
set data_call 1 apn_name=testapn1 ip_ver=4 reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 1
```

```

add data_call 2
set      data_call      2      apn_name=testapn2      ip_ver=4      reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 2

add data_call 3
set      data_call      3      apn_name=testapn3      ip_ver=4      reconn_mode=mode1
reconn_interval=10,20,40,80,120
start data_call 3

add interface intf_data_apn1 type=data_call call_id=1
add interface intf_data_apn2 type=data_call call_id=2
add interface intf_data_apn3 type=data_call call_id=3

```

步骤六 配置转发规则。

```

add acl acl_app_server sintf=intf_local url=test.com dintf=intf_data_apn3
add acl acl_app_apn1 sintf=intf_local dintf=intf_data_apn1
add acl acl_lan42_server sintf=intf_lan42 url=test.com dintf=intf_data_apn3
add acl acl_lan41_apn1 sintf=intf_lan41 dintf=intf_data_apn1
add acl acl_lan42_apn2 sintf=intf_lan42 dintf=intf_data_apn2
add acl acl_lan43_apn3 sintf=intf_lan43 dintf=intf_data_apn3

```

5 附录 参考文档及术语缩写

表 20: 参考文档

文档名称
[1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导
[2] Quectel_EC200A-CN(TA)_QuecOpen_Wi-Fi_API_参考手册
[3] Quectel_EC200A-CN(TA)_QuecOpen_数据拨号 API_参考手册

表 21: 术语缩写

缩写	英文描述	中文描述
ACL	Access Control Lists	访问控制列表
AP	Access Point	无线接入点
APP	Application	应用程序
APN	Access Point Name	接入点名称
CHAP	Challenge Handshake Authentication Protocol	挑战握手认证协议
DHCP	Dynamic Host Configuration Protocol	动态主机配置协议
DNS	Domain Name System	域名系统
DSDA	Dual SIM Dual Active	双卡双通
ECL	Emitter Coupled Logic	发射极耦合逻辑电路
ETH	Ethernet	以太网
LAN	Local Area Network	局域网
LTE	Long Term Evolution	长期演进

ID	Identification	标识
IoT	Internet of Things	物联网
IP	Internet Protocol	互联网协议
ISP	Internet Service Provider	互联网服务提供商
MAC	Media Access Control	媒体存取控制位
MTU	Maximum Transmission Unit	最大传输单元
NCM	Network Control Model	网络控制模型
OUI	Organizationally Unique Identifier	组织唯一标识符
PAP	Password Authentication Protocol	密码认证协议
PC	Personal Computer	个人计算机
PPP	Point to Point Protocol	点对点协议
QOS	Quality of Service	服务质量
QL_NF	QUECTEL Network Framework	移远通信网络管理框架
RNDIS	Remote Network Driver Interface Specification	远程网络驱动程序接口规范
SDK	Software Development Kit	软件开发工具包
SIM	Subscriber Identity Modula	用户识别卡
TCP	Transmission Control Protocol	传输控制协议
UDP	User Datagram Protocol	用户数据报协议
URL	Uniform Resource Locator	统一资源定位符
USB	Universal Serial Bus	通用串行总线
VLAN	Virtual Local Area Network	虚拟局域网
VPN	Virtual Private Network	虚拟专用网络
WAN	Wide Area Network	广域网
WLAN	Wireless Local Area Network	无线局域网
WIFI	Wireless Fidelity	无线通信技术